

chihalo's R notebook

Chihalo Kamanga

Contents

Preface	5
What's in here	5
How to read this book	5
Acknowledgements	5
1 Data Visualisation	7
1.1 Introduction	7
1.2 First steps	7
Exercises	11
1.3 ggplot2 calls	17
1.4 Visualizing distributions	18
1.4.1 A categorical variable	18
1.4.2 A numerical variable	19
Exercises	21
1.5 Visualizing relationships	24
1.5.1 A numerical and a categorical variable	24
1.5.2 Two categorical variables	25
1.5.3 Two numerical variables	27
1.5.4 Three or more variables	28
Exercises	29
1.6 Saving your plots	35
2 Workflow: basics	37
2.1 Coding basics	37
2.2 Comments	37
2.3 What's in a name?	37
2.4 Calling functions	38
2.5 Exercises	38
3 Data transformation	41
3.1 Introduction	41
3.1.1 Prerequisites	41
3.1.2 nycflights13	41
3.1.3 dplyr basics	43
3.2 Rows	44
3.2.1 filter()	44
3.2.2 Common mistakes	49
3.2.3 arrange()	49
3.2.4 distinct()	50
3.2.5 Exercises	54
3.3 Columns	60
3.3.1 mutate()	60

3.3.2	<code>select()</code>	63
3.3.3	<code>rename()</code>	66
3.3.4	<code>relocate()</code>	66
3.3.5	Exercises	68
3.4	The pipe	72
3.5	Groups	72
3.5.1	<code>group_by()</code>	72
3.5.2	<code>summarize()</code>	73
3.5.3	The <code>slice_</code> functions	75
3.5.4	Grouping by multiple variables	77
3.5.5	Ungrouping	78
3.5.6	<code>.by</code>	79
3.5.7	Exercises	80
3.6	Case study: aggregates and sample size	86
4	Workflow: code style	89
4.1	Names	89
4.2	Spaces	89
4.3	Pipes	90
4.4	ggplot2	90
4.5	Sectioning comments	91
4.6	Exercises	92
5	Data tidying	95
5.1	Introduction	95
5.1.1	Prerequisites	95
5.2	Tidy data	95
5.2.1	Exercises	96
5.3	Lengthening data	98
5.3.1	Data in column names	98
5.3.2	How does pivoting work?	100
6	Summary	103
	References	105
	Bibliography	107

Preface

This book is my journey and record of learning **R** using *R for Data Science (2nd edition)* by Hadley and company.

Started these notes on 20 July 2026. They began as plain R scripts that I wrote for myself as I worked through the book, and I've turned them into this Quarto book or whatever!

What's in here

So far I've worked through:

- Chapter 1.: making plots with ggplot2
- Chapter 2.: the basics of writing R code
- Chapter 3.: filtering, sorting, and summarising data with dplyr
- Chapter 4.: writing tidy, readable code
- Chapter 5.: reshaping data with tidyr (still in progress)

More chapters will be added as I go.

How to read this book

i Note

These are learning notes, not a textbook – take them with a the largest pinch of salt you ca find. They show my attempts, my mistakes, and what I worked out along the way, so expect some rough edges.

- Every code chunk is real R code, and the output you see was produced when the book was built.
- Some code is shown but not run, for example code that only works in RStudio or installs packages.
- Some code fails **on purpose**, to show what a particular error looks like.
- The exercises come from the R4DS book, and the answers are my own attempts. They may not be perfect, and corrections are welcome.

Acknowledgements

All credit belongs to *R4DS 2e*. Original book at is at r4ds.hadley.nz. One time yeah!

1. Data Visualisation

My notes and practice for learning R with the R4DS (2nd edition) book. Started 20th July, 2026.

1.1 Introduction

```
library(tidyverse)
library(palmerpenguins)
library(nycflights13)
library(ggthemes)
library(Lahman)
```

1.2 First steps

Take a first look at the data.

flights

```
# A tibble: 336,776 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     542           540           2     923           850
4  2013     1     1     544           545          -1    1004          1022
5  2013     1     1     554           600          -6     812           837
6  2013     1     1     554           558          -4     740           728
7  2013     1     1     555           600          -5     913           854
8  2013     1     1     557           600          -3     709           723
9  2013     1     1     557           600          -3     838           846
10 2013     1     1     558           600          -2     753           745
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

penguins

```
# A tibble: 344 × 8
  species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
  <fct>   <fct>         <dbl>         <dbl>         <int>         <int>
1 Adelie  Torgersen      39.1          18.7          181          3750
2 Adelie  Torgersen      39.5          17.4          186          3800
3 Adelie  Torgersen      40.3           18           195          3250
4 Adelie  Torgersen      NA            NA            NA            NA
5 Adelie  Torgersen      36.7          19.3          193          3450
6 Adelie  Torgersen      39.3          20.6          190          3650
7 Adelie  Torgersen      38.9          17.8          181          3625
8 Adelie  Torgersen      39.2          19.6          195          4675
9 Adelie  Torgersen      34.1          18.1          193          3475
10 Adelie  Torgersen      42           20.2          190          4250
```

```
# i 334 more rows
# i 2 more variables: sex <fct>, year <int>
```

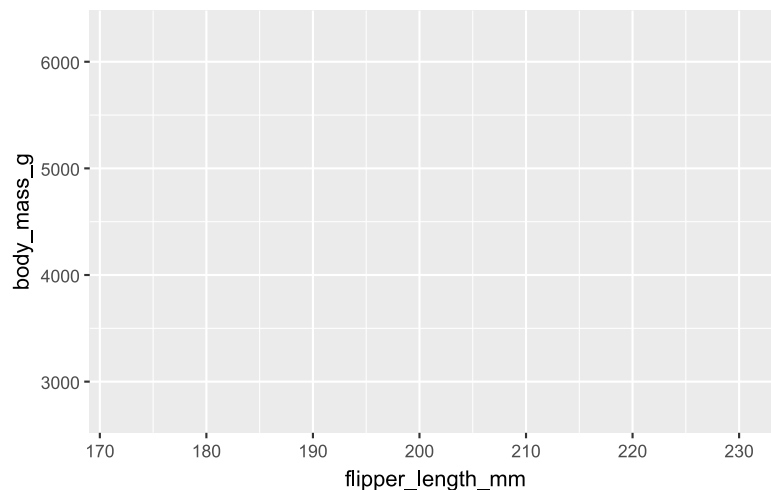
```
glimpse(penguins)
```

```
Rows: 344
Columns: 8
$ species      <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel...
$ island       <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgerse...
$ bill_length_mm <dbl> 39.1, 39.5, 40.3, NA, 36.7, 39.3, 38.9, 39.2, 34.1, ...
$ bill_depth_mm <dbl> 18.7, 17.4, 18.0, NA, 19.3, 20.6, 17.8, 19.6, 18.1, ...
$ flipper_length_mm <int> 181, 186, 195, NA, 193, 190, 181, 195, 193, 190, 186...
$ body_mass_g   <int> 3750, 3800, 3250, NA, 3450, 3650, 3625, 4675, 3475, ...
$ sex          <fct> male, female, female, NA, female, male, female, male...
$ year         <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007...
```

```
View(penguins)
?penguins
```

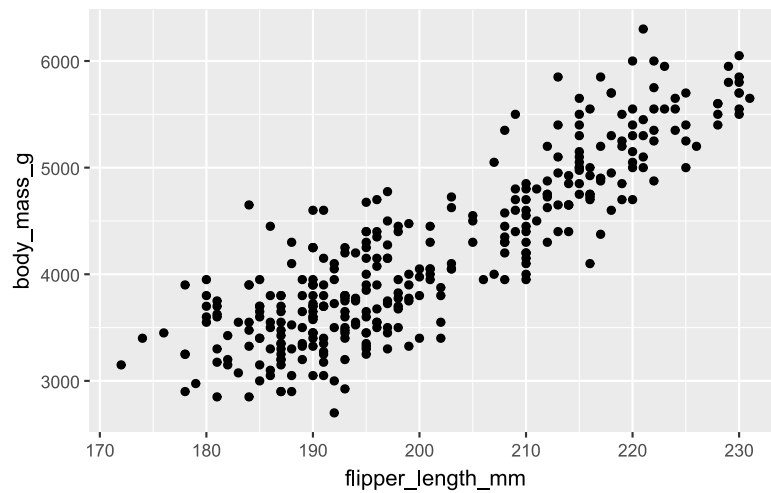
An empty plot: we only set the data and the mapping.

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g))
```



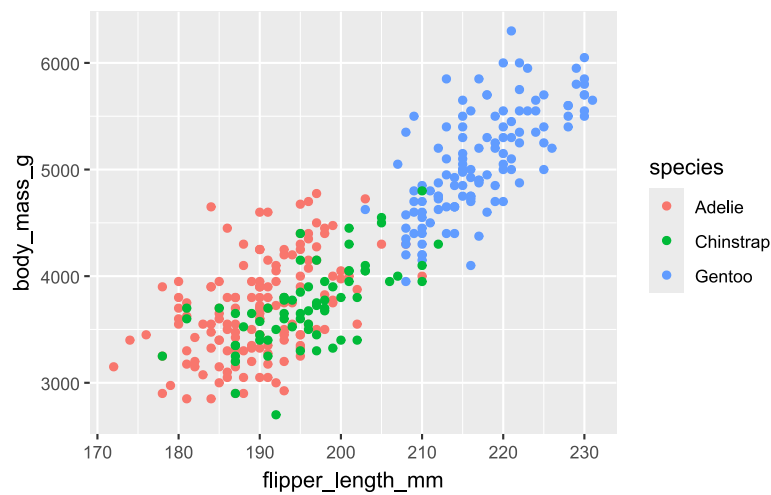
Adding a geom. Note the + that connects the geom. **Every layer must be connected with a +.**

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point()
```



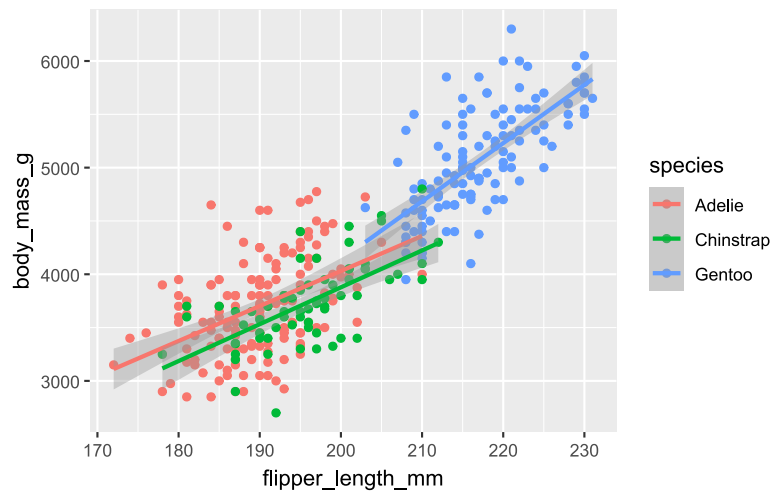
Adding an aesthetic (colour by species):

```
ggplot(  
  data = penguins,  
  mapping = aes(x = flipper_length_mm, y = body_mass_g, colour = species)  
) +  
  geom_point()
```



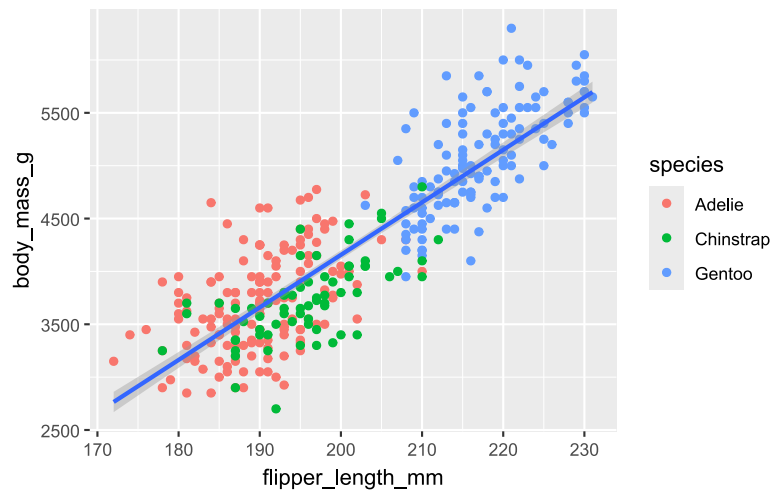
Adding another geom layer:

```
ggplot(  
  data = penguins,  
  mapping = aes(x = flipper_length_mm, y = body_mass_g, colour = species)  
) +  
  geom_point() +  
  geom_smooth(method = "lm")
```



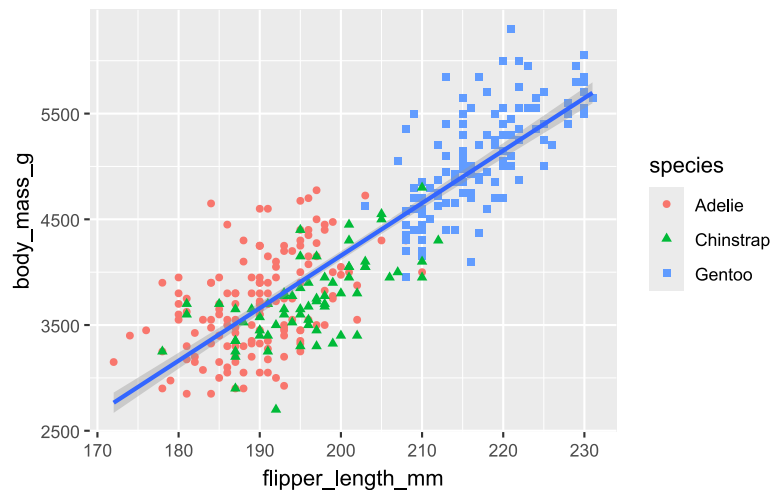
Adding an aesthetic to a specific geom. Note how `colour = species` is not in the **global** `aes()`, so there is one trend line instead of three.

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point(mapping = aes(colour = species)) +
  geom_smooth(method = "lm")
```



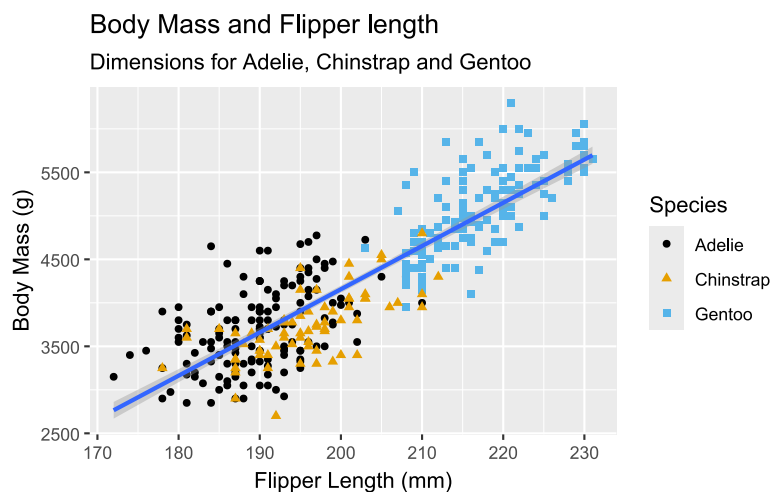
Mapping species to shape as well:

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point(mapping = aes(colour = species, shape = species)) +
  geom_smooth(method = "lm")
```



Improving the labels:

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point(mapping = aes(colour = species, shape = species)) +
  geom_smooth(method = "lm") +
  labs(
    title = "Body Mass and Flipper length",
    subtitle = "Dimensions for Adelie, Chinstrap and Gentoo",
    x = "Flipper Length (mm)",
    y = "Body Mass (g)",
    color = "Species",
    shape = "Species"
  ) +
  scale_color_colorblind()
```



Exercises

1. How many rows are in `penguins`? How many columns?

```
nrow(penguins)
```

```
[1] 344
```

```
ncol(penguins)
```

```
[1] 8
```

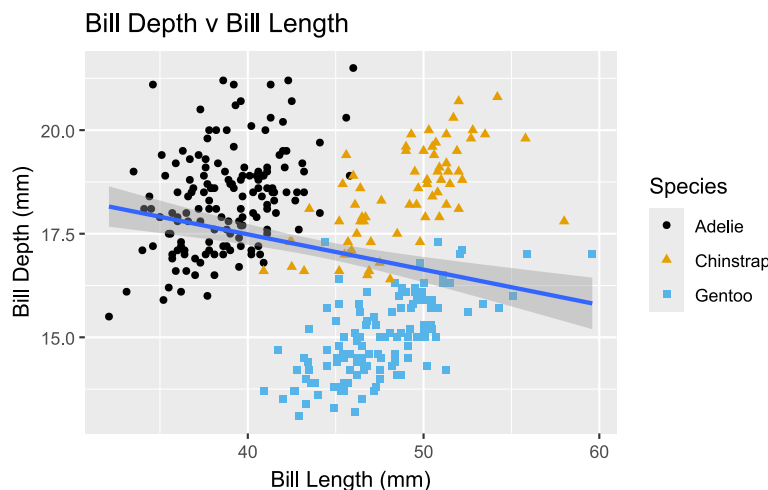
2. What does the `bill_depth_mm` variable in the `penguins` data frame describe?

Read the help page with `?penguins` to find out.

```
?penguins
```

3. Make a scatterplot of `bill_depth_mm` vs. `bill_length_mm`. Describe the relationship between these two variables.

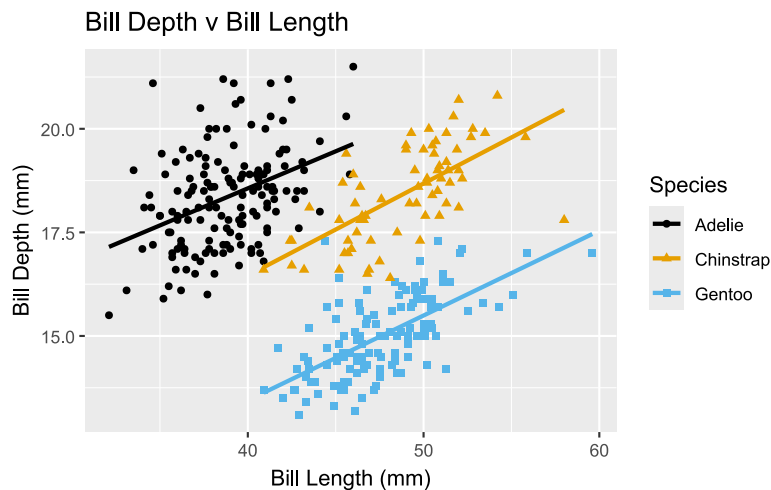
```
ggplot(data = penguins,
       mapping = aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point(mapping = aes(colour = species, shape = species)) +
  geom_smooth(method = "lm") +
  labs(
    title = "Bill Depth v Bill Length",
    x = "Bill Length (mm)",
    y = "Bill Depth (mm)",
    colour = "Species",
    shape = "Species"
  ) +
  scale_color_colorblind()
```



Overall, there is a negative relationship between the two variables, but within each species the relationship is positive.

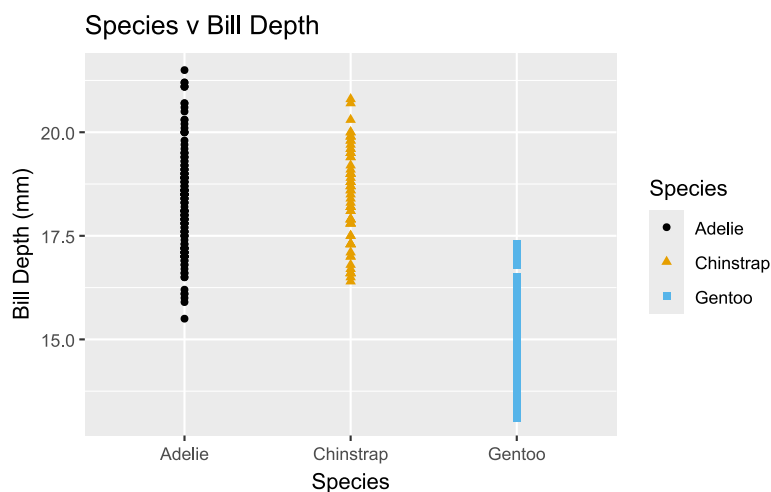
To show the positive correlation within species, move `colour = species` to the global `aes()` and add `se = FALSE` to remove the standard error shading:

```
ggplot(
  data = penguins,
  mapping = aes(color = species, x = bill_length_mm, y = bill_depth_mm)
) +
  geom_point(mapping = aes(shape = species)) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(
    title = "Bill Depth v Bill Length",
    x = "Bill Length (mm)",
    y = "Bill Depth (mm)",
    colour = "Species",
    shape = "Species"
  ) +
  scale_color_colorblind()
```



4. What happens if you make a scatterplot of `species` vs. `bill_depth_mm`?

```
ggplot(data = penguins,
       mapping = aes(x = species, y = bill_depth_mm)) +
  geom_point(mapping = aes(colour = species, shape = species)) +
  geom_smooth(method = "lm") +
  labs(
    title = "Species v Bill Depth",
    x = "Species",
    y = "Bill Depth (mm)",
    colour = "Species",
    shape = "Species"
  ) +
  scale_color_colorblind()
```

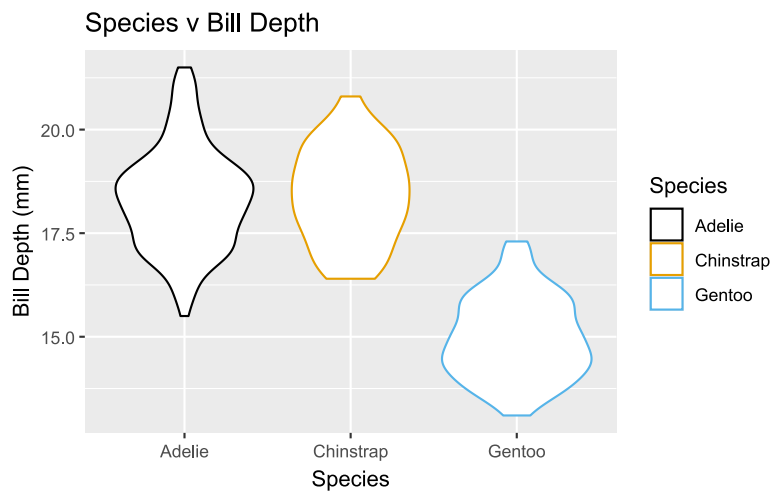


What might be a better choice of geom?

A violin plot is good because it shows the shape of the data distribution, so you can see where most values are concentrated. It is more informative than a boxplot, but you could also try the box.

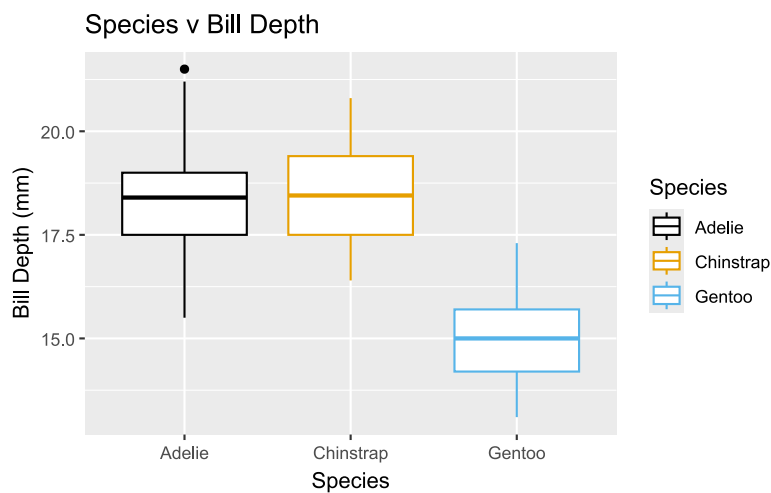
```
ggplot(data = penguins,
       mapping = aes(x = species, y = bill_depth_mm)) +
  geom_violin(mapping = aes(colour = species)) +
  labs(
    title = "Species v Bill Depth",
    x = "Species",
    y = "Bill Depth (mm)",
    colour = "Species"
  )
```

```
) +  
scale_color_colorblind()
```



Boxplot:

```
ggplot(data = penguins,  
       mapping = aes(x = species, y = bill_depth_mm)) +  
  geom_boxplot(mapping = aes(colour = species)) +  
  labs(  
    title = "Species v Bill Depth",  
    x = "Species",  
    y = "Bill Depth (mm)",  
    colour = "Species"  
  ) +  
  scale_color_colorblind()
```



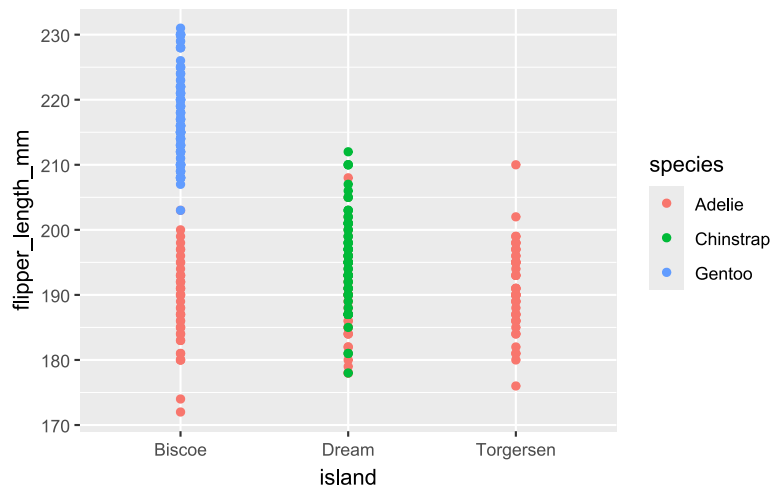
5. Why does the following give an error? How would you fix it?

```
ggplot(data = penguins) +  
  geom_point()
```

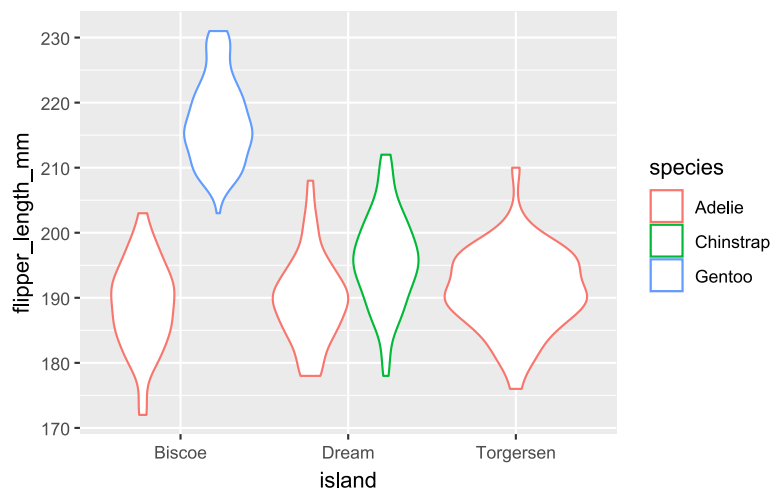
```
Error in `geom_point()` :  
! Problem while setting up geom.  
! Error occurred in the 1st layer.  
Caused by error in `compute_geom_1()` :  
! `geom_point()` requires the following missing aesthetics: x and y.
```

There is no mapping, so `geom_point()` has no `x` and `y` aesthetics to draw. The fix is to supply a mapping. With the variables I chose, `geom_point()` is not the best geom, so a boxplot or violin may be better.

```
ggplot(data = penguins,
       mapping = aes(x = island, y = flipper_length_mm)) +
  geom_point(mapping = aes(color = species))
```



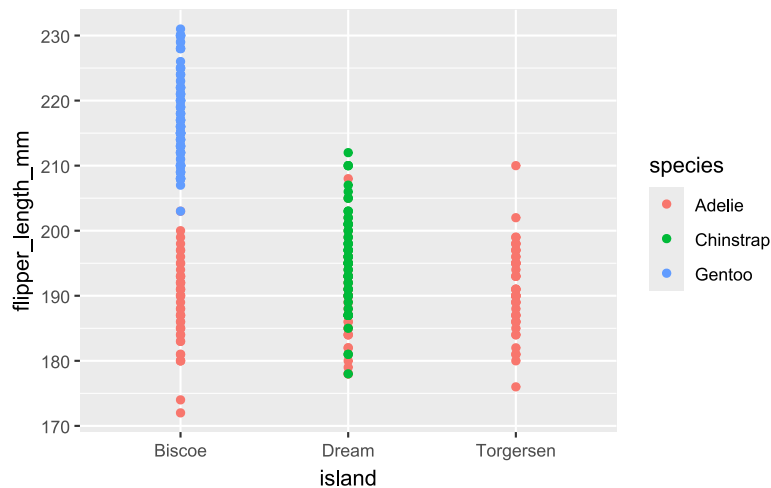
```
ggplot(data = penguins,
       mapping = aes(x = island, y = flipper_length_mm)) +
  geom_violin(mapping = aes(color = species))
```



6. What does the `na.rm` argument do in `geom_point()`? What is the default value? Create a scatterplot where you successfully use this argument set to `TRUE`.

`na.rm` controls whether missing values are removed silently. The default is `na.rm = FALSE`, which means R still removes the missing values but warns you that it did. Setting `na.rm = TRUE` removes them without the warning.

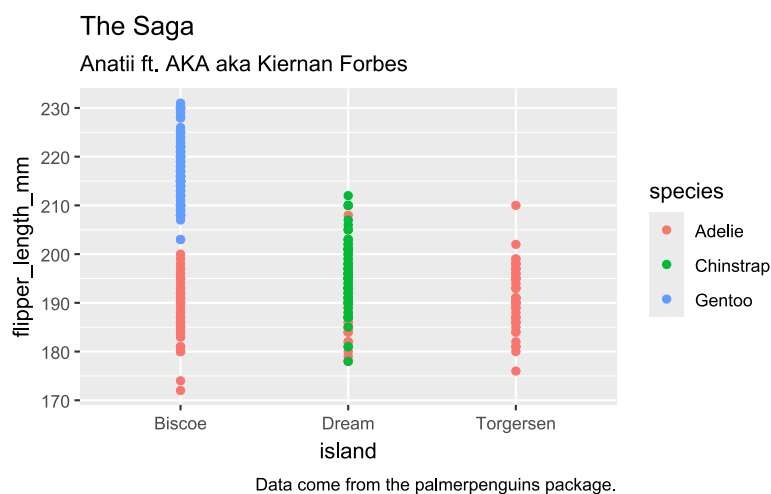
```
ggplot(data = penguins,
       mapping = aes(x = island, y = flipper_length_mm)) +
  geom_point(mapping = aes(color = species),
            na.rm = TRUE)
```



7. Add the caption “Data come from the palmerpenguins package.” to the plot from the previous exercise.

Hint: look at the documentation for `labs()`.

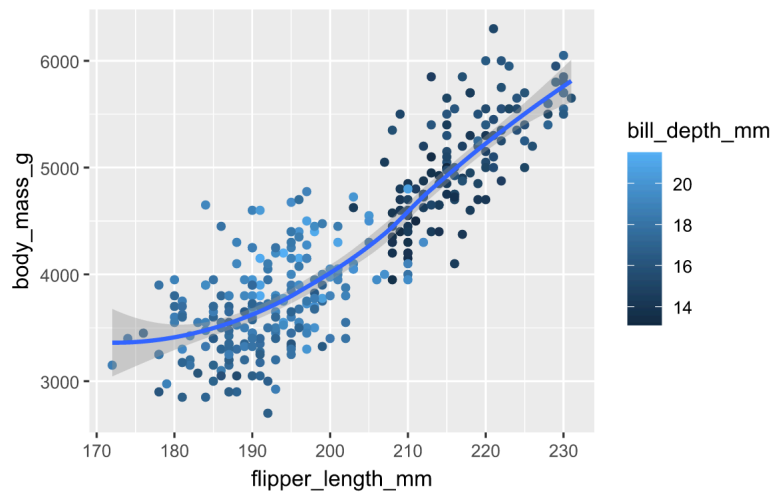
```
ggplot(data = penguins,
       mapping = aes(x = island, y = flipper_length_mm)) +
  geom_point(mapping = aes(color = species),
            na.rm = TRUE) +
  labs(caption = "Data come from the palmerpenguins package.",
       title = "The Saga",
       subtitle = "Anatii ft. AKA aka Kiernan Forbes")
```



8. Recreate the visualization from the R4DS book. What aesthetic should `bill_depth_mm` be mapped to? Should it be mapped at the global level or at the geom level?

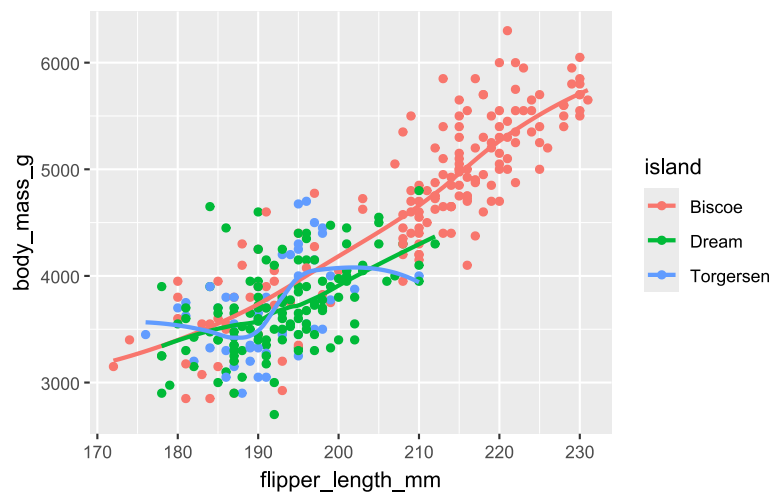
`bill_depth_mm` is mapped to `colour`, at the geom level (only the points use it, not the smooth line).

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point(mapping = aes(colour = bill_depth_mm)) +
  geom_smooth()
```



9. Run this code in your head and predict what the output will look like. Then run it in R and check your prediction.

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g, color = island)
) +
  geom_point() +
  geom_smooth(se = FALSE)
```

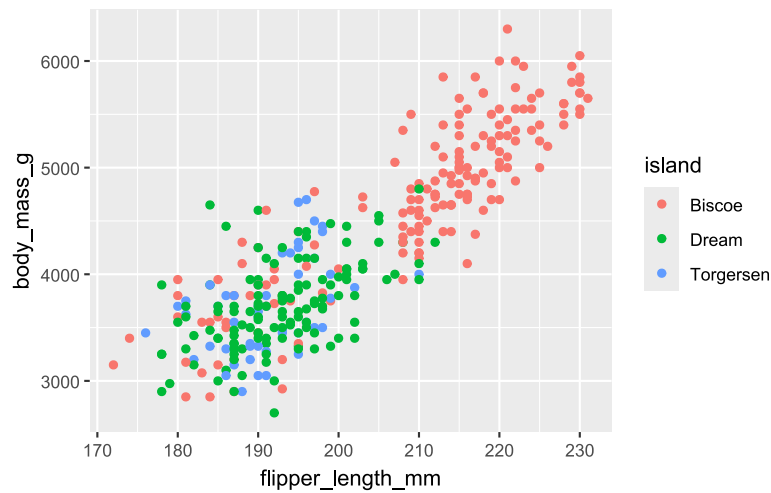


To get a single trend line, put the `color` aesthetic in the local `geom_point()` instead.

1.3 ggplot2 calls

You can leave out the explicit `data =` and `mapping =`. This is more concise:

```
ggplot(penguins, aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point(aes(color = island))
```



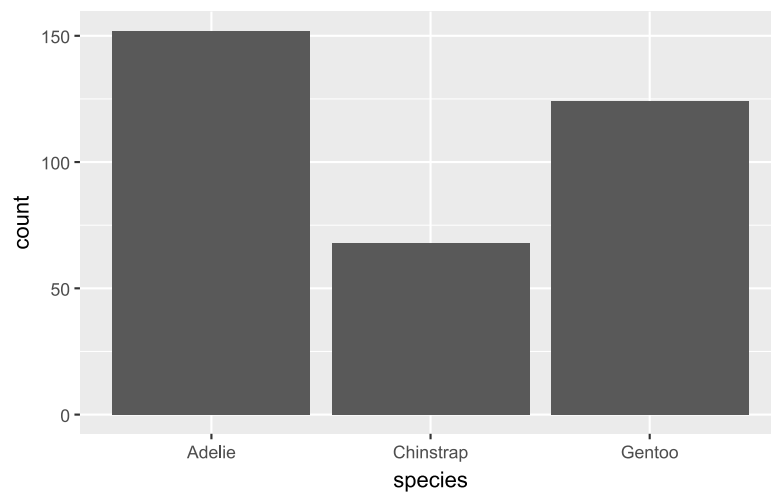
1.4 Visualizing distributions

How you visualize the distribution of a variable depends on the type of variable: categorical or numerical.

1.4.1 A categorical variable

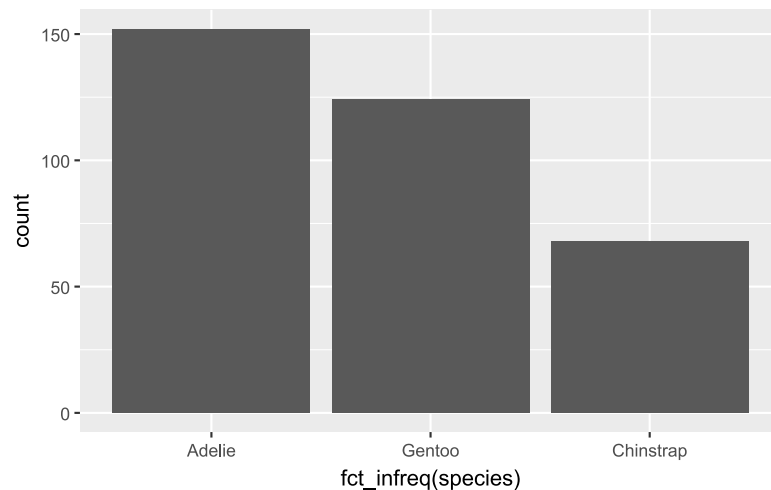
A variable is **categorical** if it can only take one of a small set of values. To see the distribution, use a bar chart:

```
ggplot(penguins, aes(x = species)) +  
  geom_bar()
```



To reorder the bars by frequency, use `fct_infreq()`. It works on categorical variables only.

```
ggplot(penguins, aes(x = fct_infreq(species))) +  
  geom_bar()
```

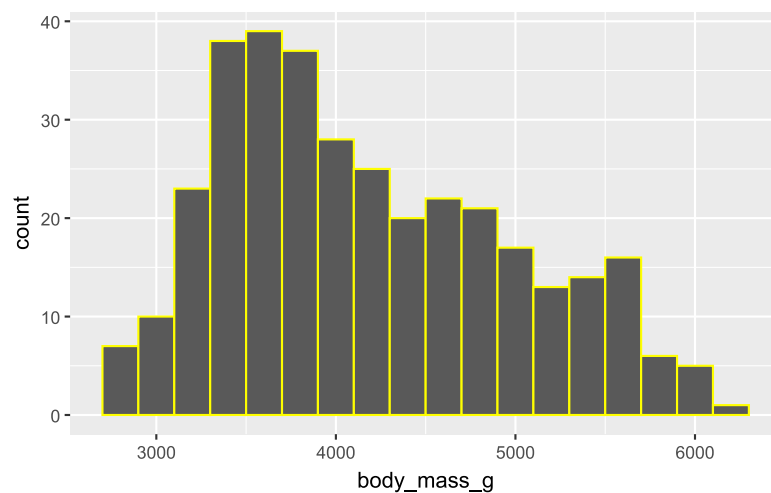


1.4.2 A numerical variable

A variable is **numerical** if it can take a wide range of numerical values, and it is sensible to add, subtract, or take averages with those values. Numerical variables can be continuous (e.g. 39.1, 65.8) or discrete (e.g. 40, 38).

A common visualization is the histogram:

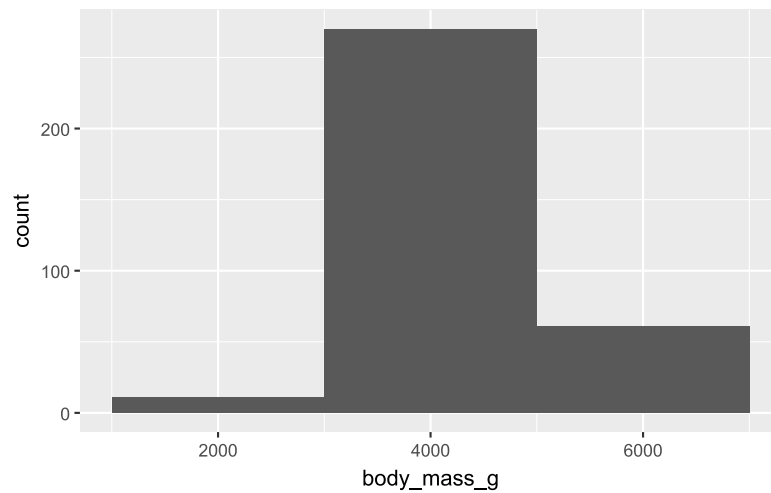
```
ggplot(penguins, aes(x = body_mass_g)) +  
  geom_histogram(binwidth = 200, colour = "yellow")
```



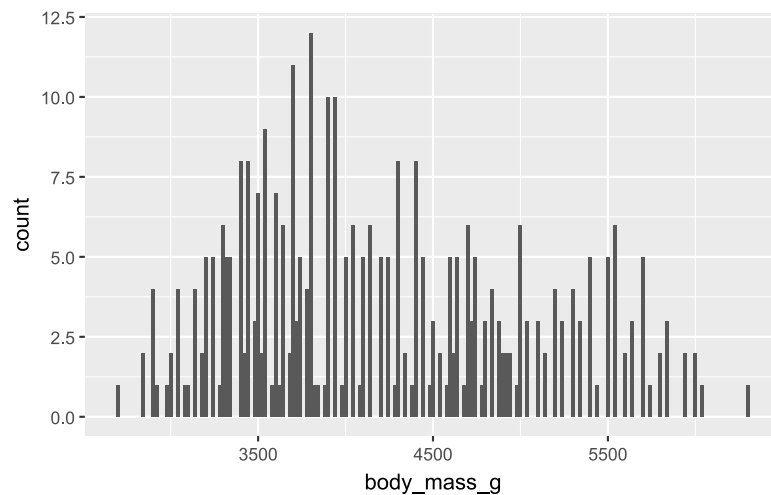
A histogram divides the x-axis into equally spaced bins and then uses the height of a bar to display the number of observations in each bin.

The binwidth matters:

```
# terrible binwidth, hides the distribution  
ggplot(penguins, aes(x = body_mass_g)) +  
  geom_histogram(binwidth = 2000)
```

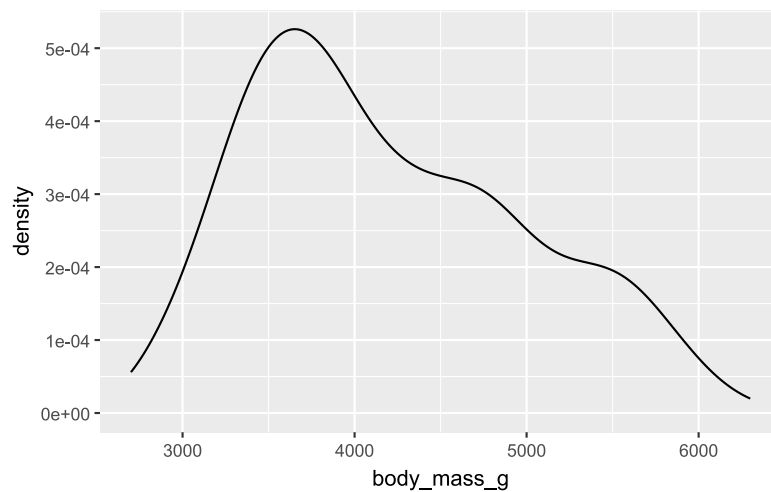


```
# not great either: too narrow, too many bars
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(binwidth = 20)
```



A density plot is an alternative to a histogram for a numerical variable:

```
ggplot(penguins, aes(x = body_mass_g)) +
  geom_density()
```

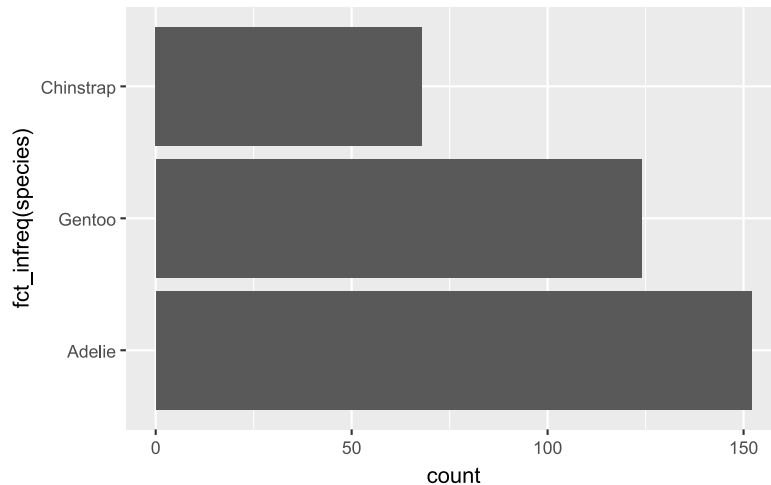


Exercises

1. Make a bar plot of **species**, where you assign **species** to the **y** aesthetic. How is this plot different?

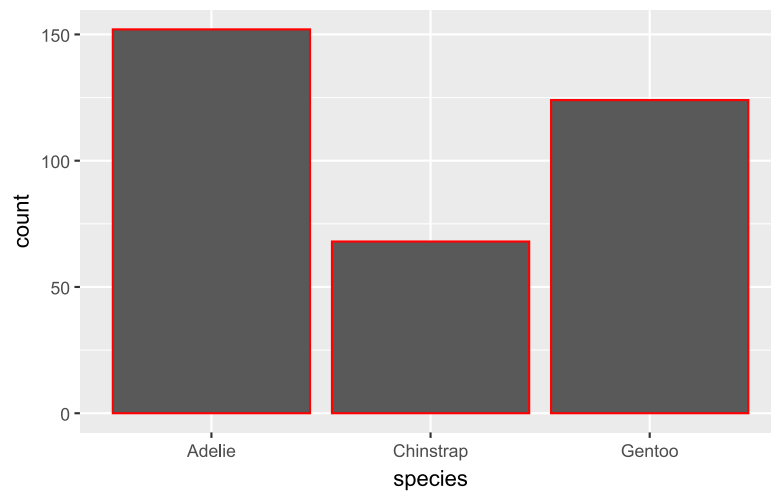
The bars flip to horizontal.

```
ggplot(penguins, aes(y = fct_infreq(species))) +  
  geom_bar()
```

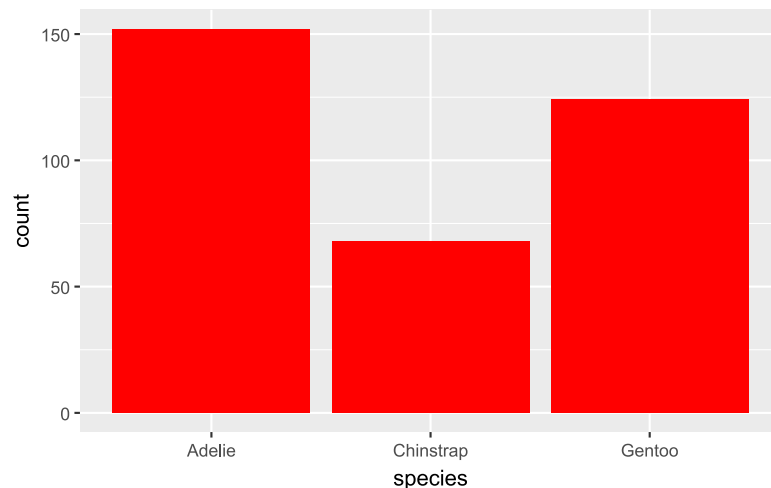


2. How are the following two plots different? Which aesthetic, **color** or **fill**, is more useful for changing the color of bars?

```
# color is for the outline  
ggplot(penguins, aes(x = species)) +  
  geom_bar(color = "red")
```



```
# fill is for the solid colour of the geom  
ggplot(penguins, aes(x = species)) +  
  geom_bar(fill = "red")
```

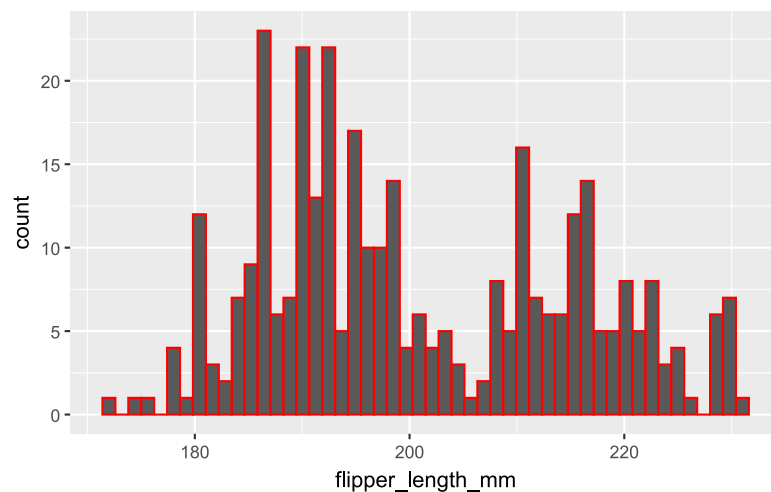


`fill` is more useful for changing the colour of bars.

3. What does the `bins` argument in `geom_histogram()` do?

It controls how many bins the data is divided into.

```
ggplot(penguins, aes(x = flipper_length_mm)) +
  geom_histogram(color = "red", bins = 50)
```



4. Make a histogram of the `carat` variable in the `diamonds` dataset. Experiment with different binwidths. What binwidth reveals the most interesting patterns?

First, explore the data:

```
View(diamonds)
```

```
summary(diamonds)
```

carat	cut	color	clarity	depth
Min. :0.2000	Fair : 1610	D: 6775	SI1 :13065	Min. :43.00
1st Qu.:0.4000	Good : 4906	E: 9797	VS2 :12258	1st Qu.:61.00
Median :0.7000	Very Good:12082	F: 9542	SI2 : 9194	Median :61.80
Mean :0.7979	Premium :13791	G:11292	VS1 : 8171	Mean :61.75
3rd Qu.:1.0400	Ideal :21551	H: 8304	VVS2 : 5066	3rd Qu.:62.50
Max. :5.0100		I: 5422	VVS1 : 3655	Max. :79.00
		J: 2808	(Other): 2531	

table	price	x	y
Min. :43.00	Min. : 326	Min. : 0.000	Min. : 0.000
1st Qu.:56.00	1st Qu.: 950	1st Qu.: 4.710	1st Qu.: 4.720

```

Median :57.00   Median : 2401   Median : 5.700   Median : 5.710
Mean   :57.46   Mean   : 3933   Mean   : 5.731   Mean   : 5.735
3rd Qu.:59.00   3rd Qu.: 5324   3rd Qu.: 6.540   3rd Qu.: 6.540
Max.   :95.00   Max.   :18823   Max.   :10.740   Max.   :58.900

```

```

      z
Min.   : 0.000
1st Qu.: 2.910
Median : 3.530
Mean   : 3.539
3rd Qu.: 4.040
Max.   :31.800

```

```
head(diamonds)
```

```

# A tibble: 6 × 10
  carat cut      color clarity depth table price     x     y     z
  <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal    E     SI2     61.5   55   326   3.95   3.98   2.43
2  0.21 Premium E     SI1     59.8   61   326   3.89   3.84   2.31
3  0.23 Good    E     VS1     56.9   65   327   4.05   4.07   2.31
4  0.29 Premium I     VS2     62.4   58   334   4.2    4.23   2.63
5  0.31 Good    J     SI2     63.3   58   335   4.34   4.35   2.75
6  0.24 Very Good J     VVS2     62.8   57   336   3.94   3.96   2.48

```

```
str(diamonds)
```

```

tibble [53,940 × 10] (S3: tbl_df/tbl/data.frame)
 $ carat  : num [1:53940] 0.23 0.21 0.23 0.29 0.31 0.24 0.24 0.26 0.22 0.23 ...
 $ cut    : Ord.factor w/ 5 levels "Fair"<"Good"<...: 5 4 2 4 2 3 3 3 1 3 ...
 $ color  : Ord.factor w/ 7 levels "D"<"E"<"F"<"G"<...: 2 2 2 6 7 7 6 5 2 5 ...
 $ clarity: Ord.factor w/ 8 levels "I1"<"SI2"<"SI1"<...: 2 3 5 4 2 6 7 3 4 5 ...
 $ depth  : num [1:53940] 61.5 59.8 56.9 62.4 63.3 62.8 62.3 61.9 65.1 59.4 ...
 $ table  : num [1:53940] 55 61 65 58 58 57 57 55 61 61 ...
 $ price  : int [1:53940] 326 326 327 334 335 336 336 337 337 338 ...
 $ x      : num [1:53940] 3.95 3.89 4.05 4.2 4.34 3.94 3.95 4.07 3.87 4 ...
 $ y      : num [1:53940] 3.98 3.84 4.07 4.23 4.35 3.96 3.98 4.11 3.78 4.05 ...
 $ z      : num [1:53940] 2.43 2.31 2.31 2.63 2.75 2.48 2.47 2.53 2.49 2.39 ...

```

```
colnames(diamonds)
```

```

[1] "carat" "cut"    "color"  "clarity" "depth"  "table"  "price"
[8] "x"     "y"     "z"

```

To see the range of the carat values:

```
summary(diamonds$carat)
```

```

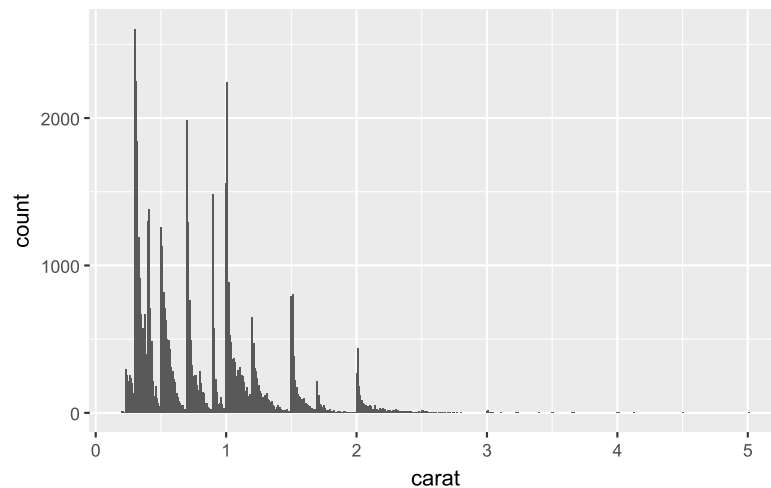
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.2000  0.4000  0.7000  0.7979  1.0400  5.0100

```

```

ggplot(diamonds, aes(x = carat)) +
  geom_histogram(binwidth = 0.01)

```



I think a binwidth of 0.01 is the best.

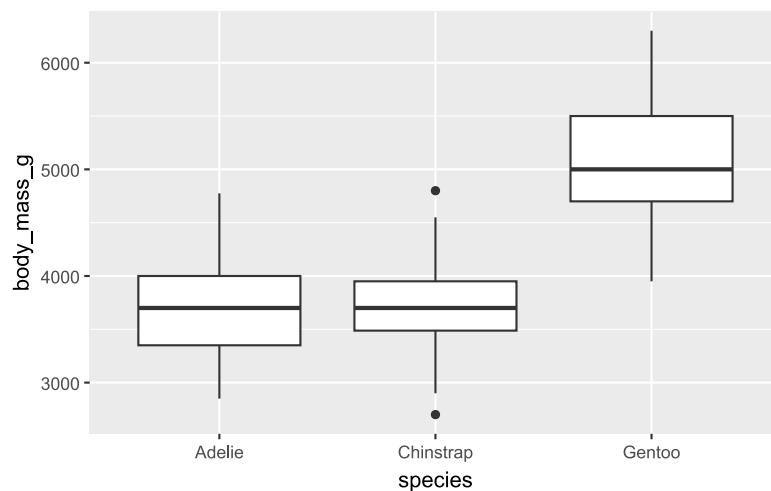
1.5 Visualizing relationships

To visualize a relationship, you need at least two variables mapped to aesthetics of a plot.

1.5.1 A numerical and a categorical variable

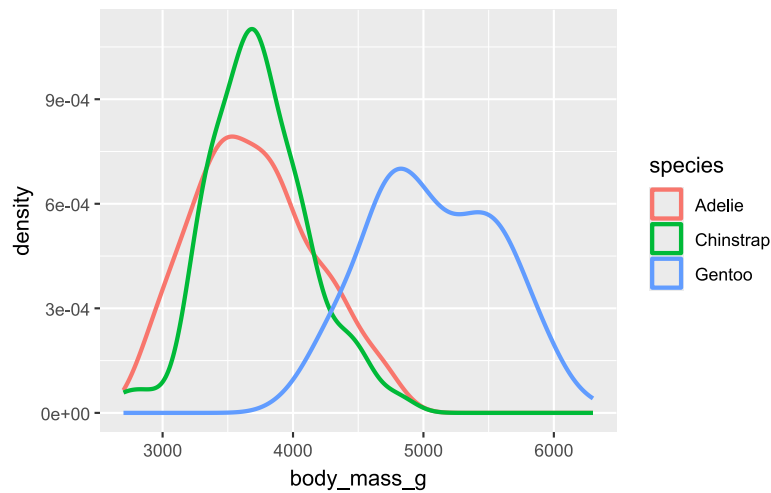
The distribution of body mass by species:

```
ggplot(penguins, aes(x = species, y = body_mass_g)) +
  geom_boxplot()
```



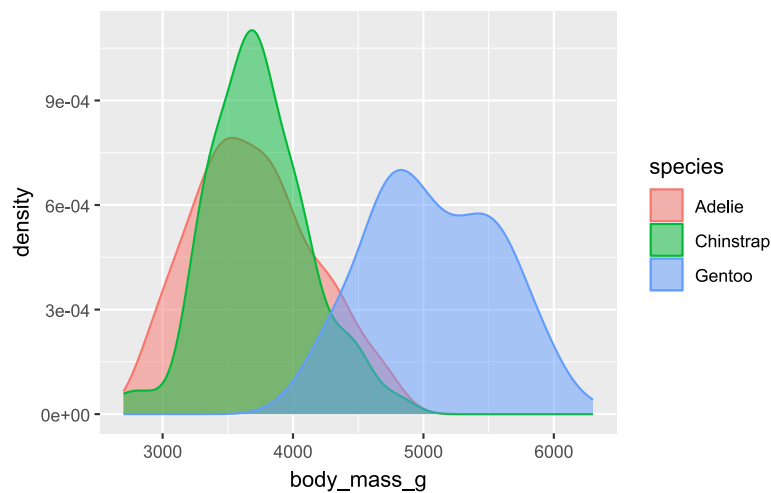
Or a density plot. Here we also customize the thickness of the lines with `linewidth` so they stand out more against the background.

```
ggplot(penguins, aes(x = body_mass_g, colour = species)) +
  geom_density(linewidth = 1)
```



Map species to both `color` and `fill`, and use the `alpha` aesthetic to add transparency. It takes values between 0 (completely transparent) and 1 (completely opaque).

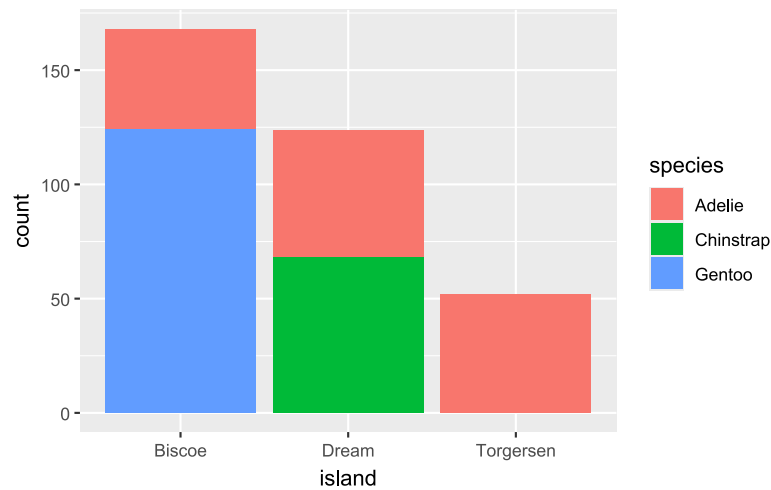
```
ggplot(penguins, aes(x = body_mass_g,
                     fill = species,
                     colour = species)) +
  geom_density(alpha = 0.5)
```



1.5.2 Two categorical variables

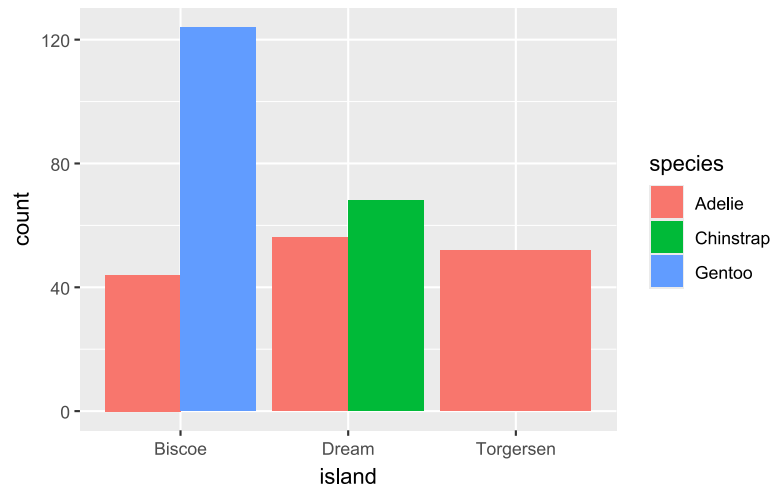
Stacked bar plots show the relationship between two categorical variables:

```
ggplot(penguins, aes(x = island, fill = species)) +
  geom_bar()
```



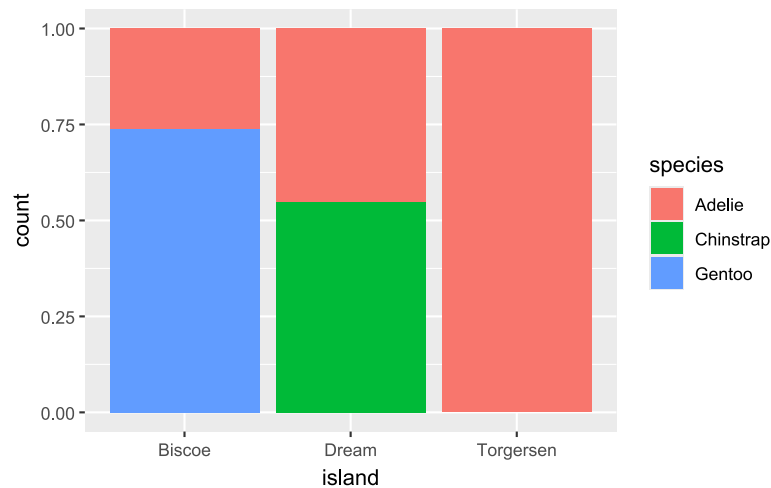
For side-by-side bars:

```
ggplot(penguins, aes(x = island, fill = species)) +  
  geom_bar(position = "dodge")
```



To compare proportions rather than raw counts (this might be better):

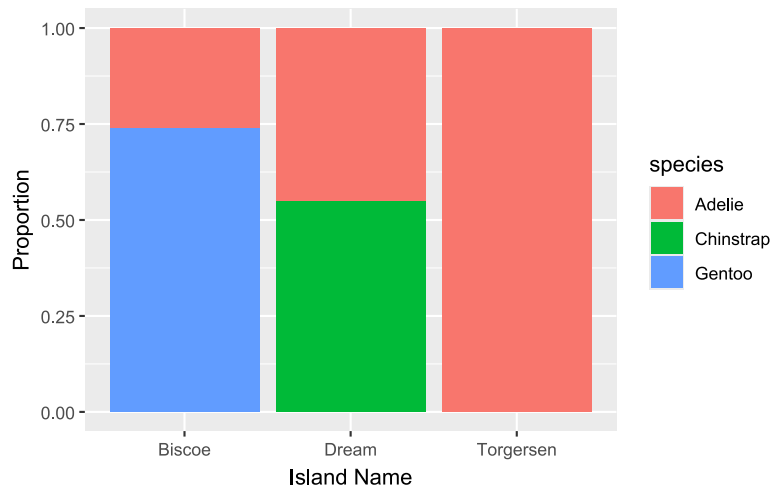
```
ggplot(penguins, aes(x = island, fill = species)) +  
  geom_bar(position = "fill")
```



In these bar charts, the variable that will be separated into bars is mapped to the `x` aesthetic, and the variable that changes the colours inside the bars is mapped to the `fill` aesthetic.

Overriding the default “count” y label:

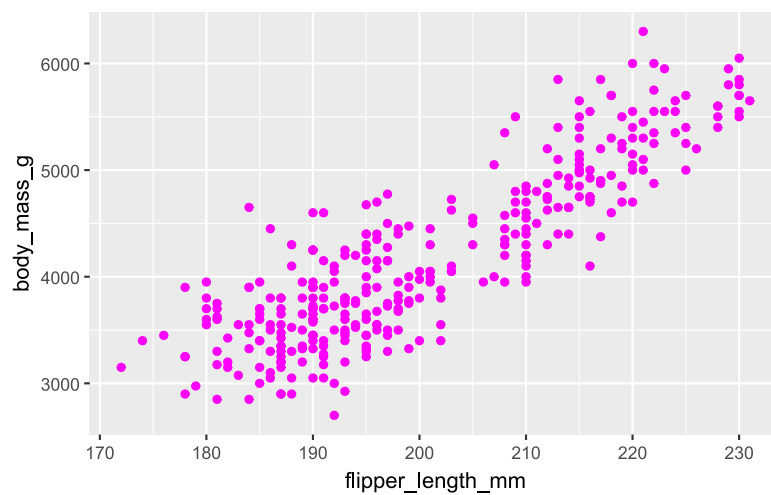
```
ggplot(penguins, aes(x = island, fill = species)) +
  geom_bar(position = "fill") +
  labs(y = "Proportion", x = "Island Name")
```



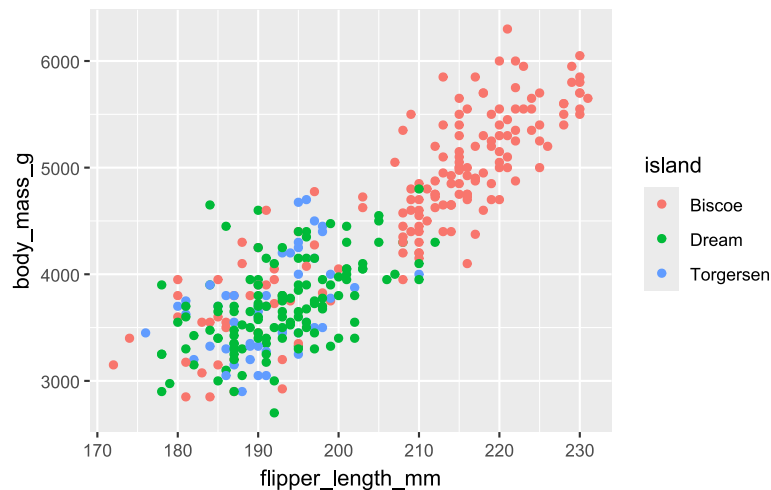
1.5.3 Two numerical variables

A scatterplot is the most common choice:

```
ggplot(penguins, aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point(colour = "magenta")
```



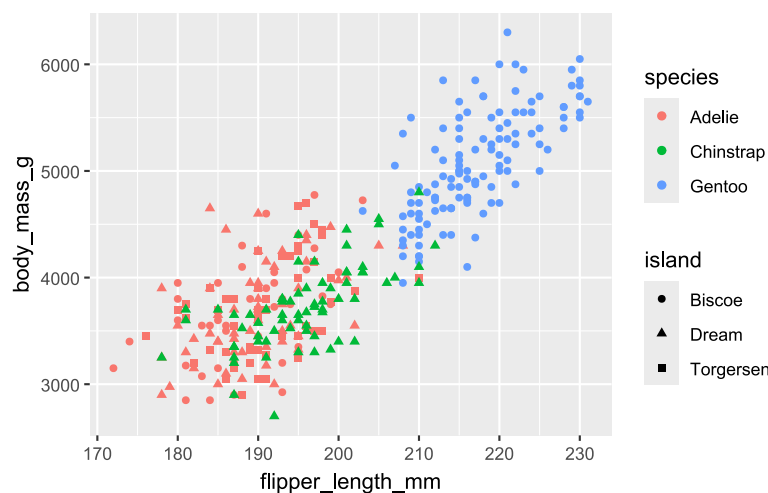
```
ggplot(penguins, aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point(aes(colour = island))
```



1.5.4 Three or more variables

We can include more variables by mapping them to additional aesthetics. In this scatterplot, the colour of points represents species and the shape represents island.

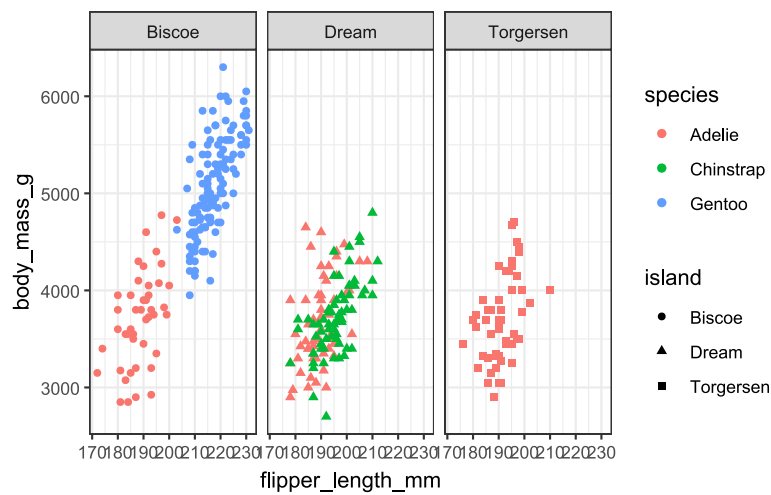
```
ggplot(penguins, aes(x = flipper_length_mm, y = body_mass_g)) +  
  geom_point(aes(colour = species, shape = island))
```



Too many aesthetics can clutter the plot, so we can use **facets** instead. Facets split the plot into subplots that each display one subset of the data. This works especially well for categorical variables.

To facet by a single variable, use `facet_wrap()`. Its first argument is a formula, created with `~` followed by a variable name. The variable you pass to `facet_wrap()` should be categorical.

```
ggplot(penguins, aes(x = flipper_length_mm, y = body_mass_g)) +  
  geom_point(aes(colour = species, shape = island)) + # you can drop the shape aes  
  facet_wrap(~island) +  
  theme_bw() # just learnt of themes, yay!
```

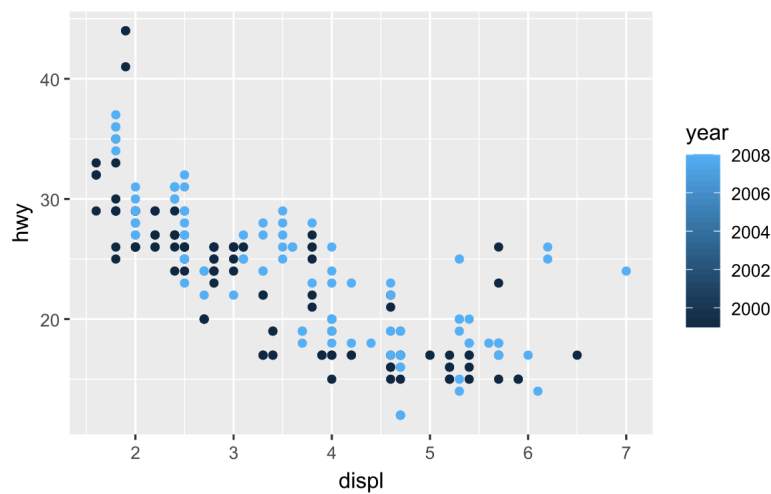


Exercises

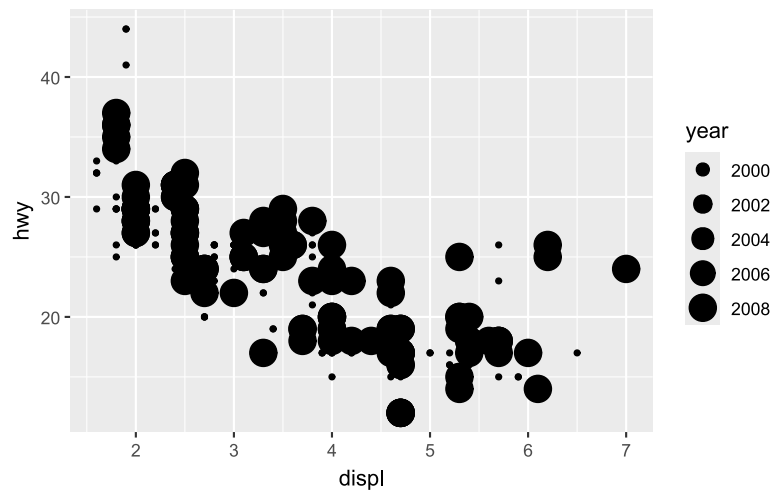
[View\(mpg\)](#)

1. Make a scatterplot of **hwy** vs. **displ** using the **mpg** data frame. Next, map a third, numerical variable to **color**, then **size**, then both **color** and **size**, then **shape**. How do these aesthetics behave differently for categorical vs. numerical variables?

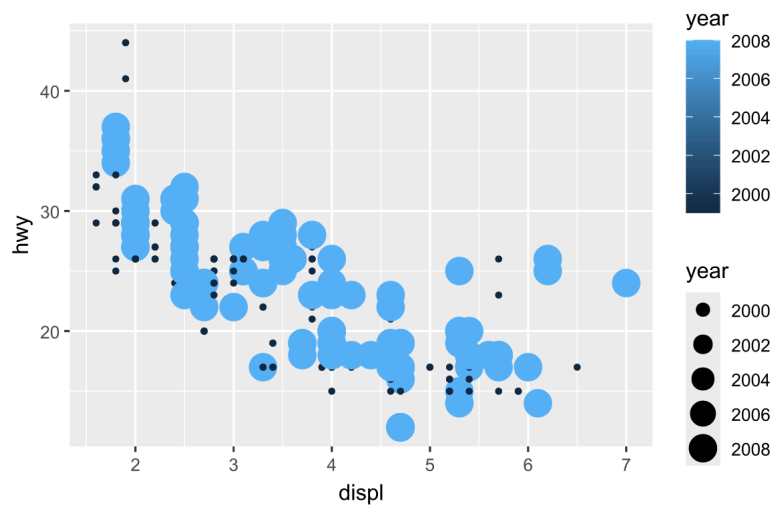
```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(aes(colour = year))
```



```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(aes(size = year))
```



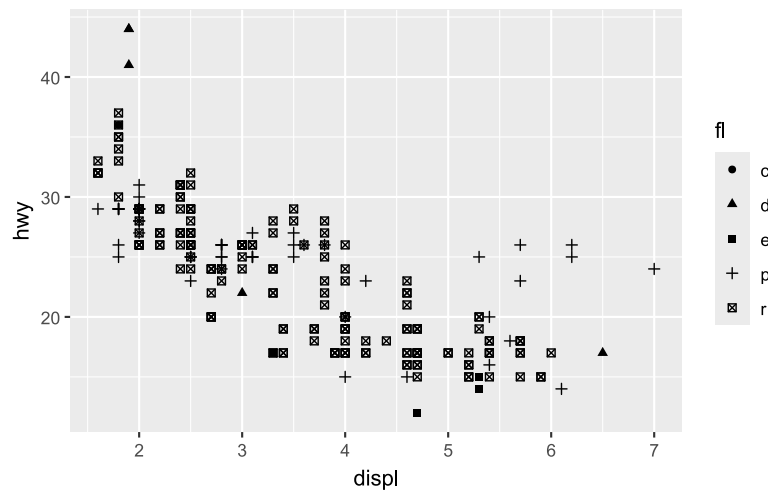
```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(aes(colour = year, size = year))
```



```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(aes(shape = year))
```

```
Error in `geom_point()`:  
! Problem while computing aesthetics.  
i Error occurred in the 1st layer.  
Caused by error in `scale_f()`:  
! A continuous variable cannot be mapped to the shape aesthetic.  
i Choose a different aesthetic or use `scale_shape_binned()`.
```

```
ggplot(mpg, aes(displ, hwy)) +  
  geom_point(aes(shape = fl))
```



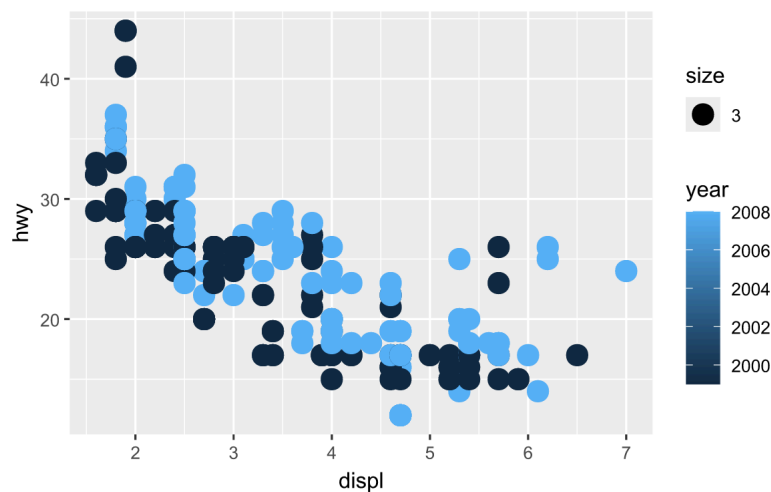
For **numerical** variables, **colour** gives a gradient (light to dark), **size** gives small to large circles, and **shape** gives an error.

For **categorical** variables, **colour** gives distinct separate colours, **size** works but looks odd, and **shape** gives different symbols like circles, triangles and squares.

Numbers have order and magnitude, so aesthetics that can show order (colour gradient, size) work well. Categories are just labels with no order, so aesthetics that show distinct differences (separate colours, shapes) work better.

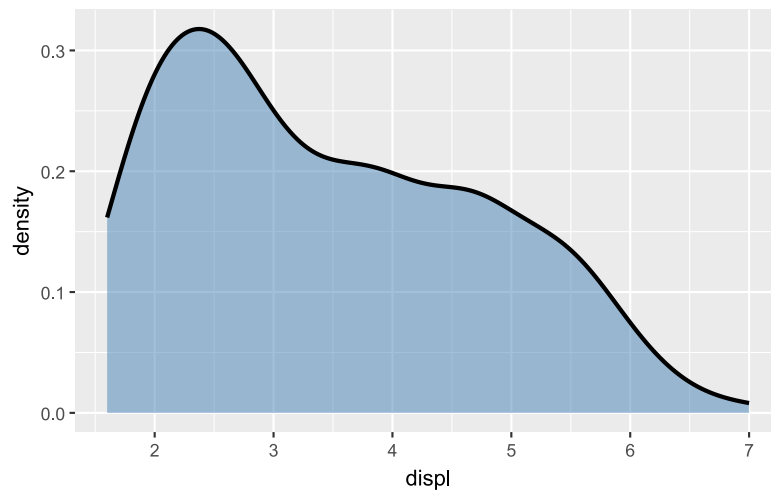
2. In the scatterplot of **hwy** vs. **displ**, what happens if you map a third variable to **linewidth**?

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(
    colour = year,
    linewidth = year,
    size = 3
  ))
```



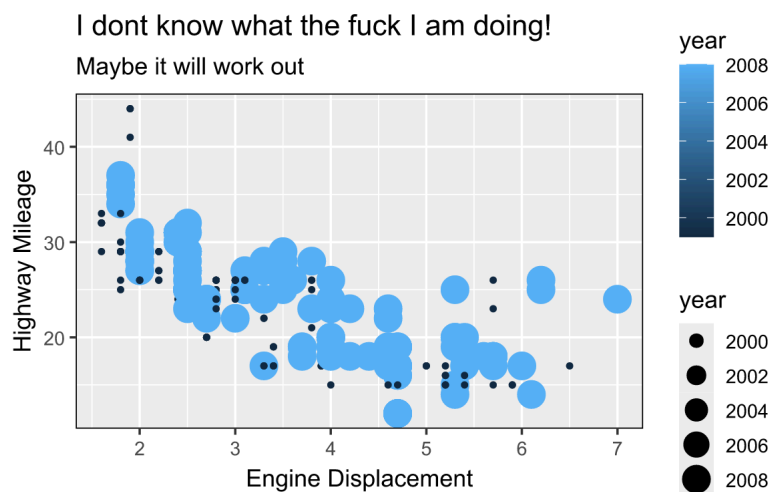
It won't throw an error, but `geom_point()` has no line width to change. I'll try a density plot instead, where `linewidth` does something:

```
ggplot(mpg, aes(displ)) +
  geom_density(linewidth = 1,
    alpha = 0.5,
    fill = "steelblue")
```



3. What happens if you map the same variable to multiple aesthetics?

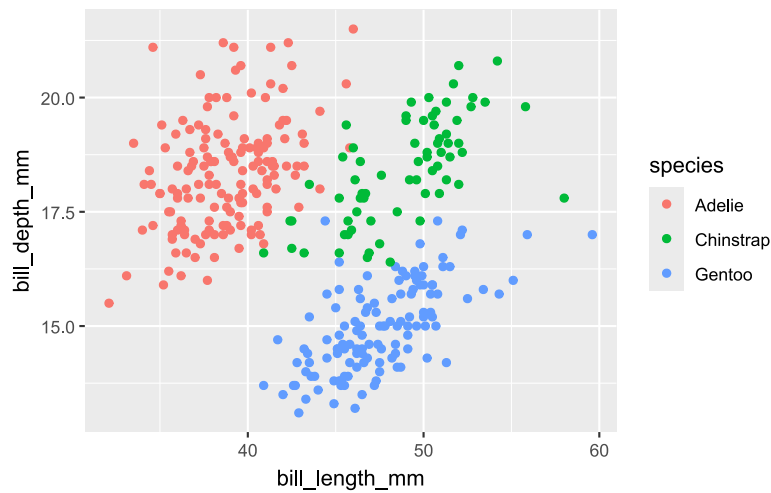
```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(colour = year, size = year)) +
  labs(
    x = "Engine Displacement",
    y = "Highway Mileage",
    title = "I dont know what the fuck I am doing!",
    subtitle = "Maybe it will work out"
  ) +
  theme_foundation()
```



It works even better: the two aesthetics reinforce each other, making the pattern even more obvious to the reader.

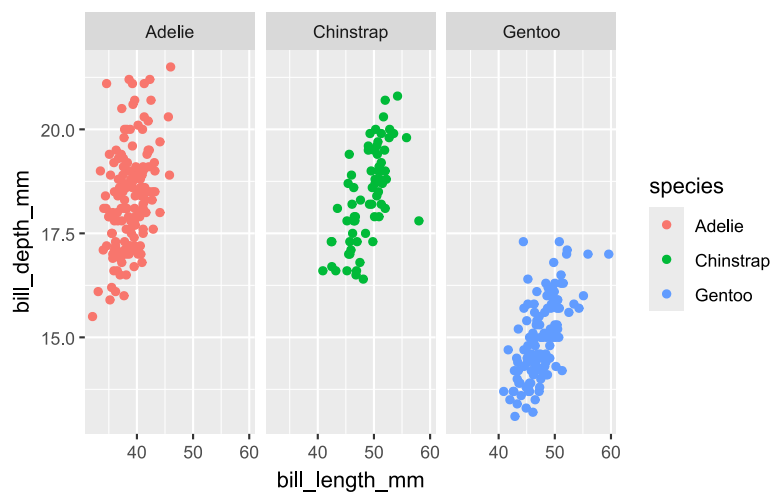
4. Make a scatterplot of `bill_depth_mm` vs. `bill_length_mm` and color the points by `species`. What does adding colouring by species reveal about the relationship between these two variables? What about faceting by species?

```
ggplot(penguins, aes(bill_length_mm, bill_depth_mm)) +
  geom_point(aes(colour = species))
```



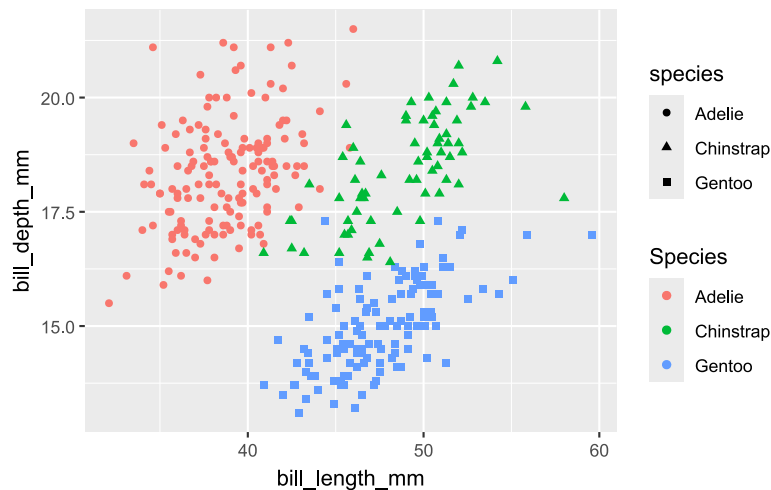
Colour reveals a positive relationship within each species, while overall the trend is flat.

```
ggplot(penguins, aes(bill_length_mm, bill_depth_mm)) +
  geom_point(aes(colour = species)) +
  facet_wrap(~species)
```



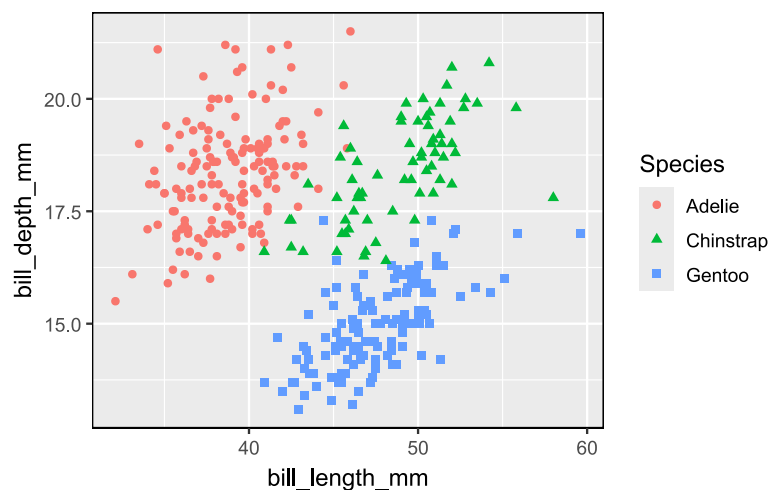
5. Why does the following yield two separate legends? How would you fix it to combine the two legends?

```
ggplot(
  data = penguins,
  mapping = aes(
    x = bill_length_mm,
    y = bill_depth_mm,
    color = species,
    shape = species
  )
) +
  geom_point() +
  labs(color = "Species")
```



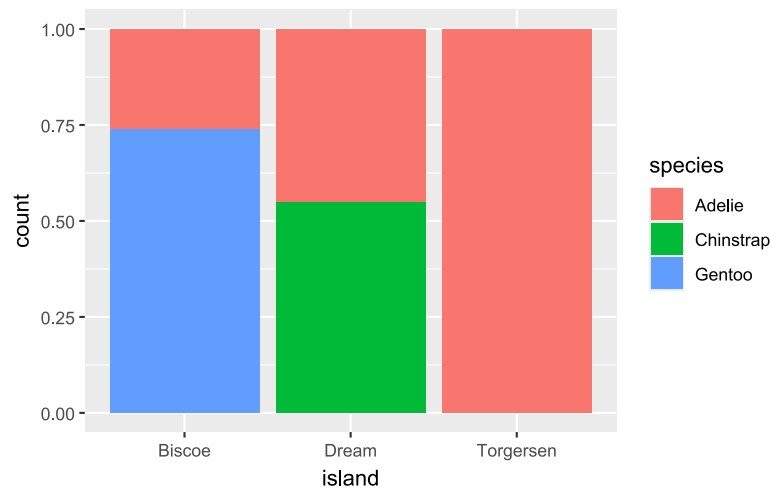
Only the `color` legend was renamed, so the `color` and `shape` legends have different titles and ggplot2 cannot merge them. The fix is to rename both legends to match:

```
ggplot(
  data = penguins,
  mapping = aes(
    x = bill_length_mm,
    y = bill_depth_mm,
    color = species,
    shape = species
  )
) +
  geom_point() +
  labs(color = "Species", shape = "Species") +
  theme_foundation()
```



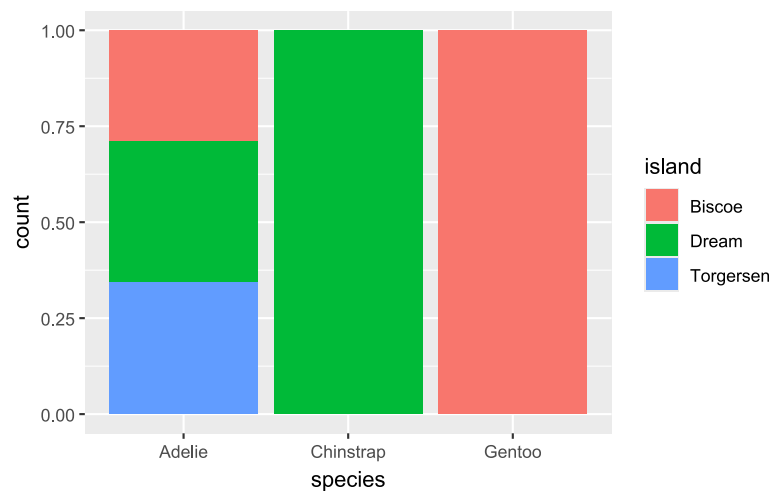
6. Create the two following stacked bar plots. Which question can you answer with the first one? Which question can you answer with the second one?

```
ggplot(penguins, aes(x = island, fill = species)) +
  geom_bar(position = "fill")
```



This answers the question: “What is the species composition of each island?”

```
ggplot(penguins, aes(x = species, fill = island)) +  
  geom_bar(position = "fill")
```



This answers the question: “Where does each species live?”

1.6 Saving your plots

Use `ggsave()`. By default, it saves the most recently created plot.

```
ggplot(penguins, aes(flipper_length_mm, bill_length_mm)) +  
  geom_point(aes(colour = species))  
  
ggsave(filename = "penguin-plot.pdf") # check your working directory
```


2. Workflow: basics

2.1 Coding basics

Using R to do basic math calculations:

```
1 / 200 * 30
```

```
[1] 0.15
```

```
(59 + 323 + 2) / 3
```

```
[1] 128
```

```
sin(pi / 3)
```

```
[1] 0.8660254
```

Create new objects with the assignment operator `<-` (shortcut: `Alt + -`):

```
x <- 3 * 5
```

Note that the value of `x` is not printed, it's just stored. To view the value, type `x` in the console.

```
x
```

```
[1] 15
```

You can combine multiple elements into a vector with `c()`:

```
primes <- c(2, 3, 5, 7, 11, 13)
```

Basic arithmetic on vectors is applied to every element of the vector:

```
primes * 2
```

```
[1] 4 6 10 14 22 26
```

```
primes - 1
```

```
[1] 1 2 4 6 10 12
```

All R statements where you create objects (assignment statements) have the same form:

```
object_name <- 58
```

2.2 Comments

Use comments to explain the **why** of your code, not the how or the what.

2.3 What's in a name?

Object names must start with a letter and can only contain letters, numbers, `_` and `..`

```
i_use_snake_case
otherPeopleUseCamelCase
some.people.use.periods
And_aFew.People.RENOUNCEconvention
```

snake_case is better: lowercase only, with _ between words.

To inspect an object, type its name:

```
x
```

```
[1] 15
```

```
this_is_a_long_ass_name <- 2.6
```

To inspect a long name, type the first few letters (like “this”) and press Tab to autocomplete.

Another assignment:

```
r_rocks <- 3 * 3
```

2.4 Calling functions

R has built-in functions that are called like this:

```
function_name(argument1 = value1, argument2 = value2, ...)
```

```
seq(from = 1, to = 10)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

The names of the first several arguments are often omitted, so we can rewrite this as:

```
seq(1, 10)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
x <- "hello world"
x
```

```
[1] "hello world"
```

2.5 Exercises

1. Why does this code not work?

```
my_variable <- 10
my_variable
```

```
Error:
! object 'my_variable' not found
```

The second line has a typo: the “i” in `variable` is a different character (a dotless i), so R looks for an object that doesn’t exist.

2. Tweak each of the following R commands so that they run correctly.

The broken versions:

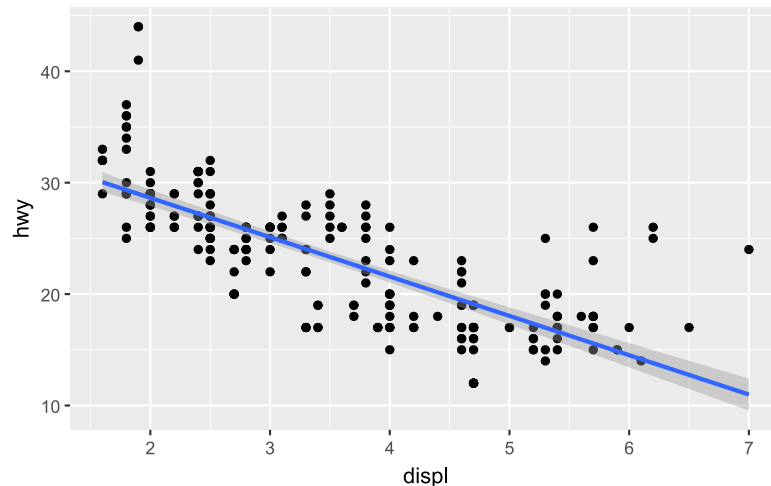
```
library(todyverse)
```

```
ggplot(dTA = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  geom_smooth(method = "lm")
```

The tweaked versions:

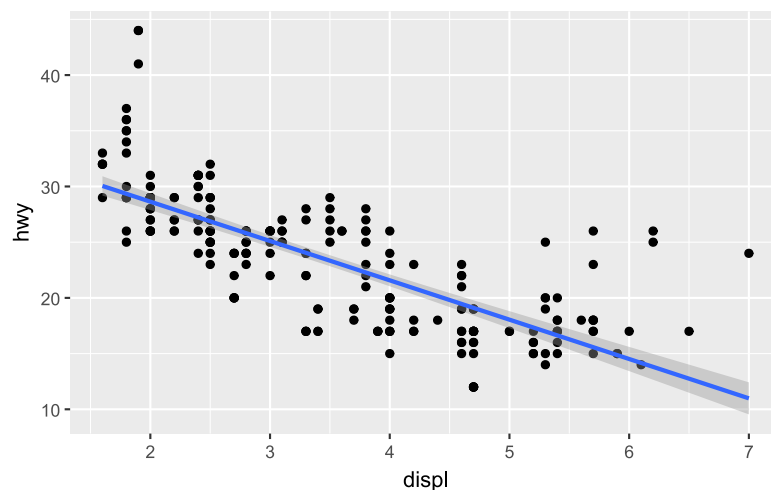
```
library(tidyverse)

ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth(method = "lm")
```



But I can write it more concisely (at my level, lol):

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point() +
  geom_smooth(method = "lm")
```



3. Press Option + Shift + K / Alt + Shift + K. What happens? How can you get to the same place using the menus?

The keyboard shortcut quick reference opens. Using the menus: Help → Keyboard Shortcuts Help.

4. Revisit an exercise from Section 1.6. Run the following lines of code. Which of the two plots is saved as `mpg-plot.png`? Why?

```
my_bar_plot <- ggplot(mpg, aes(x = class)) +
  geom_bar()
```

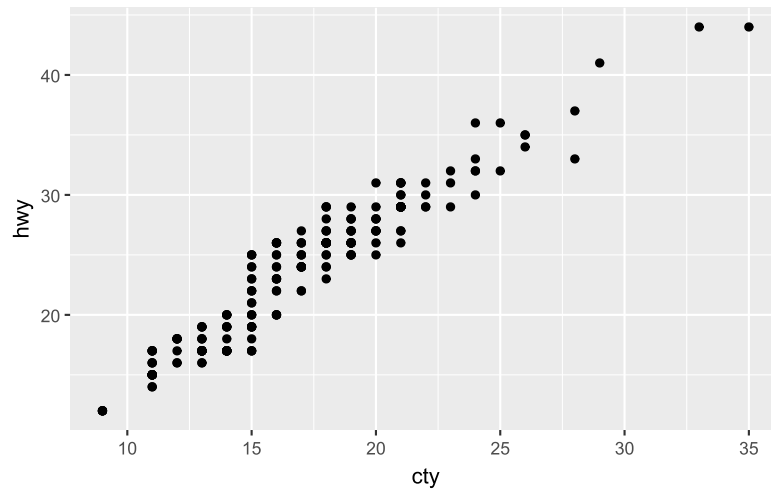
```
my_scatter_plot <- ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point()
```

```
ggsave(filename = "mpg-plot.png", plot = my_bar_plot)
```

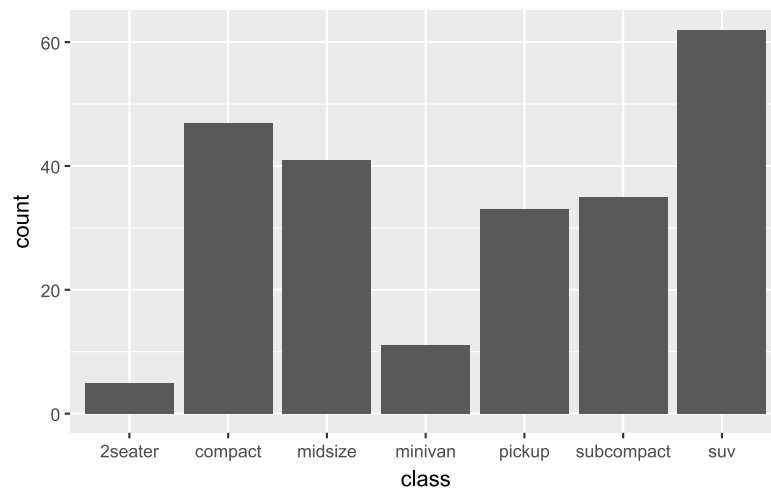
The bar plot is saved, because `plot = my_bar_plot` tells `ggsave()` exactly which plot to use.

Note that when a plot is assigned to an object, it is stored but **not shown**. Call the object by name to see it:

```
my_scatter_plot
```



```
my_bar_plot
```



3. Data transformation

3.1 Introduction

3.1.1 Prerequisites

We'll be using **dplyr** with **nycflights13**, a package with multiple datasets: `flights`, `weather`, `airlines`, `airports` and `planes`.

If you haven't installed `nycflights13` yet, run this once in the Console:

```
install.packages("nycflights13")
```

```
library(tidyverse)
library(nycflights13)
library(palmerpenguins)
```

3.1.2 nycflights13

```
nrow(flights)
```

```
[1] 336776
```

```
head(flights)
```

```
# A tibble: 6 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2     830             819
2  2013     1     1     533             529           4     850             830
3  2013     1     1     542             540           2     923             850
4  2013     1     1     544             545          -1    1004            1022
5  2013     1     1     554             600          -6     812             837
6  2013     1     1     554             558          -4     740             728
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
head(weather)
```

```
# A tibble: 6 × 15
  origin year month   day hour temp dewp humid wind_dir wind_speed wind_gust
  <chr>  <int> <int> <int> <int> <dbl> <dbl> <dbl>   <dbl>    <dbl>    <dbl>
1 EWR    2013     1     1     1 39.0 26.1 59.4    270    10.4      NA
2 EWR    2013     1     1     2 39.0 27.0 61.6    250     8.06     NA
3 EWR    2013     1     1     3 39.0 28.0 64.4    240    11.5      NA
4 EWR    2013     1     1     4 39.9 28.0 62.2    250    12.7      NA
5 EWR    2013     1     1     5 39.0 28.0 64.4    260    12.7      NA
6 EWR    2013     1     1     6 37.9 28.0 67.2    240    11.5      NA
# i 4 more variables: precip <dbl>, pressure <dbl>, visib <dbl>,
#   time_hour <dtm>
```

```
head(planes)
```

```
# A tibble: 6 × 9
  tailnum year type      manufacturer model engines seats speed engine
  <chr>   <int> <chr>      <chr>      <chr>   <int> <int> <int> <chr>
1 N10156  2004 Fixed wing multi ... EMBRAER    EMB-...     2    55    NA Turbo...
2 N102UW  1998 Fixed wing multi ... AIRBUS INDU... A320...     2   182    NA Turbo...
3 N103US  1999 Fixed wing multi ... AIRBUS INDU... A320...     2   182    NA Turbo...
4 N104UW  1999 Fixed wing multi ... AIRBUS INDU... A320...     2   182    NA Turbo...
5 N10575  2002 Fixed wing multi ... EMBRAER    EMB-...     2    55    NA Turbo...
6 N105UW  1999 Fixed wing multi ... AIRBUS INDU... A320...     2   182    NA Turbo...
```

```
View(flights)
```

```
?flights
```

```
flights
```

```
# A tibble: 336,776 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2     830             819
2  2013     1     1     533             529           4     850             830
3  2013     1     1     542             540           2     923             850
4  2013     1     1     544             545          -1    1004            1022
5  2013     1     1     554             600          -6     812             837
6  2013     1     1     554             558          -4     740             728
7  2013     1     1     555             600          -5     913             854
8  2013     1     1     557             600          -3     709             723
9  2013     1     1     557             600          -3     838             846
10 2013     1     1     558             600          -2     753             745
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
glimpse(flights)
```

```
Rows: 336,776
Columns: 19
$ year      <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2...
$ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
$ day       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
$ dep_time  <int> 517, 533, 542, 544, 554, 554, 555, 557, 557, 558, 558, ...
$ sched_dep_time <int> 515, 529, 540, 545, 600, 558, 600, 600, 600, 600, 600, ...
$ dep_delay <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -3, -2, -2, -2, -2, -2, -1...
$ arr_time  <int> 830, 850, 923, 1004, 812, 740, 913, 709, 838, 753, 849,...
$ sched_arr_time <int> 819, 830, 850, 1022, 837, 728, 854, 723, 846, 745, 851,...
$ arr_delay <dbl> 11, 20, 33, -18, -25, 12, 19, -14, -8, 8, -2, -3, 7, -1...
$ carrier   <chr> "UA", "UA", "AA", "B6", "DL", "UA", "B6", "EV", "B6", "...
$ flight    <int> 1545, 1714, 1141, 725, 461, 1696, 507, 5708, 79, 301, 4...
$ tailnum   <chr> "N14228", "N24211", "N619AA", "N804JB", "N668DN", "N394...
$ origin    <chr> "EWR", "LGA", "JFK", "JFK", "LGA", "EWR", "EWR", "LGA",...
$ dest      <chr> "IAH", "IAH", "MIA", "BQN", "ATL", "ORD", "FLL", "IAD",...
$ air_time  <dbl> 227, 227, 160, 183, 116, 150, 158, 53, 140, 138, 149, 1...
$ distance  <dbl> 1400, 1416, 1089, 1576, 762, 719, 1065, 229, 944, 733, ...
$ hour      <dbl> 5, 5, 5, 5, 6, 5, 6, 6, 6, 6, 6, 6, 6, 6, 5, 6, 6, 6...
$ minute    <dbl> 15, 29, 40, 45, 0, 58, 0, 0, 0, 0, 0, 0, 0, 0, 59, 0...
$ time_hour <dtm> 2013-01-01 05:00:00, 2013-01-01 05:00:00, 2013-01-01 0...
```

Setting `width = Inf` basically tells R “I don’t care how wide it is, show me everything”:

```
print(flights, width = Inf)
```

```
# A tibble: 336,776 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2       830           819
2  2013     1     1     533             529           4       850           830
3  2013     1     1     542             540           2       923           850
4  2013     1     1     544             545          -1      1004          1022
5  2013     1     1     554             600          -6       812           837
6  2013     1     1     554             558          -4       740           728
7  2013     1     1     555             600          -5       913           854
8  2013     1     1     557             600          -3       709           723
9  2013     1     1     557             600          -3       838           846
10 2013     1     1     558             600          -2       753           745
  arr_delay carrier flight tailnum origin dest air_time distance hour minute
    <dbl>   <chr>   <int> <chr>   <chr> <chr>   <dbl>   <dbl> <dbl> <dbl>
1      11    UA     1545 N14228  EWR   IAH     227    1400     5     15
2      20    UA     1714 N24211  LGA   IAH     227    1416     5     29
3      33    AA     1141 N619AA  JFK   MIA     160    1089     5     40
4     -18    B6       725 N804JB  JFK   BQN     183    1576     5     45
5     -25    DL       461 N668DN  LGA   ATL     116     762     6      0
6      12    UA     1696 N39463  EWR   ORD     150     719     5     58
7      19    B6       507 N516JB  EWR   FLL     158    1065     6      0
8     -14    EV     5708 N829AS  LGA   IAD      53     229     6      0
9      -8    B6        79 N593JB  JFK   MCO     140     944     6      0
10     8    AA       301 N3ALAA  LGA   ORD     138     733     6      0
  time_hour
  <dtm>
1 2013-01-01 05:00:00
2 2013-01-01 05:00:00
3 2013-01-01 05:00:00
4 2013-01-01 05:00:00
5 2013-01-01 06:00:00
6 2013-01-01 05:00:00
7 2013-01-01 06:00:00
8 2013-01-01 06:00:00
9 2013-01-01 06:00:00
10 2013-01-01 06:00:00
# i 336,766 more rows
```

- `width` = how wide the output is allowed to be
- `Inf` = infinity = no limit

3.1.3 dplyr basics

`dplyr` uses verbs (functions). They differ, but have these things in common:

1. The first argument is always a data frame.
2. The subsequent arguments typically describe which columns to operate on, using the variable names (without quotes).
3. The output is always a new data frame.

Complex problems require multiple verbs, so we use the pipe (`|>`). In brief, the pipe takes the thing on its left and passes it along to the function on its right.

An example (I am not familiar with this yet, lol):

```
flights |>
  filter(dest == "IAH") |>
  group_by(year, month, day) |>
```

```
summarize(
  arr_delay = mean(arr_delay, na.rm = TRUE)
)
```

```
# A tibble: 365 × 4
# Groups:   year, month [12]
   year month   day arr_delay
  <int> <int> <int>    <dbl>
1  2013     1     1     17.8
2  2013     1     2       7
3  2013     1     3     18.3
4  2013     1     4     -3.2
5  2013     1     5     20.2
6  2013     1     6      9.28
7  2013     1     7     -7.74
8  2013     1     8      7.79
9  2013     1     9     18.1
10 2013     1    10      6.68
# i 355 more rows
```

The same idea for Miami:

```
flights |>
  filter(dest == "MIA") |>
  group_by(year, month, day) |>
  summarize(
    arr_delay = mean(arr_delay, na.rm = TRUE)
  )
```

```
# A tibble: 365 × 4
# Groups:   year, month [12]
   year month   day arr_delay
  <int> <int> <int>    <dbl>
1  2013     1     1      5.8
2  2013     1     2      1.13
3  2013     1     3      9.29
4  2013     1     4     -5.69
5  2013     1     5     -11.6
6  2013     1     6      1.23
7  2013     1     7     -7.34
8  2013     1     8     -1.16
9  2013     1     9     -12.3
10 2013     1    10     -17.8
# i 355 more rows
```

dplyr's verbs are organized into four groups based on what they operate on:

- rows
- columns
- tables
- groups

3.2 Rows

3.2.1 filter()

`filter()` keeps rows based on the values of the columns. It lets you say “I only want to see rows where a certain condition is true” and throws away all the rest.

The first argument is always the data frame. The second and later arguments are the conditions that must be true. For instance, flights that departed more than 120 minutes late:

```
flights |>
  filter(dep_delay > 120)
```

```
# A tibble: 9,723 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     848           1835         853    1001          1950
2  2013     1     1     957           1733         144    1056          853
3  2013     1     1    1114           1900         134    1447         1222
4  2013     1     1    1540           1338         122    2020         1825
5  2013     1     1    1815           1325         290    2120         1542
6  2013     1     1    1842           1422         260    1958         1535
7  2013     1     1    1856           1645         131    2212         2005
8  2013     1     1    1934           1725         129    2126         1855
9  2013     1     1    1938           1703         155    2109         1823
10 2013     1     1    1942           1705         157    2124         1830
# i 9,713 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

As well as > (greater than), you can use >= (greater than or equal to), < (less than), <= (less than or equal to), == (equal to), and != (not equal to).

You can also combine conditions with & or , to mean “and” (check for both conditions), or with | to mean “or” (check for either condition).

Flights that departed on January 1:

```
flights |>
  filter(month == 1, day == 1)
```

```
# A tibble: 842 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515          2      830          819
2  2013     1     1     533           529          4      850          830
3  2013     1     1     542           540          2      923          850
4  2013     1     1     544           545         -1     1004         1022
5  2013     1     1     554           600         -6      812          837
6  2013     1     1     554           558         -4      740          728
7  2013     1     1     555           600         -5      913          854
8  2013     1     1     557           600         -3      709          723
9  2013     1     1     557           600         -3      838          846
10 2013     1     1     558           600         -2      753          745
# i 832 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

or

```
flights |>
  filter(month == 1 & day == 1)
```

```
# A tibble: 842 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515          2      830          819
2  2013     1     1     533           529          4      850          830
3  2013     1     1     542           540          2      923          850
4  2013     1     1     544           545         -1     1004         1022
```

```

5 2013 1 1 554 600 -6 812 837
6 2013 1 1 554 558 -4 740 728
7 2013 1 1 555 600 -5 913 854
8 2013 1 1 557 600 -3 709 723
9 2013 1 1 557 600 -3 838 846
10 2013 1 1 558 600 -2 753 745
# i 832 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

Flights that departed in January or February (| means “or”):

```

flights |>
  filter(month == 1 | month == 2)

```

```

# A tibble: 51,955 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     542           540           2     923           850
4  2013     1     1     544           545          -1    1004          1022
5  2013     1     1     554           600          -6     812           837
6  2013     1     1     554           558          -4     740           728
7  2013     1     1     555           600          -5     913           854
8  2013     1     1     557           600          -3     709           723
9  2013     1     1     557           600          -3     838           846
10 2013     1     1     558           600          -2     753           745
# i 51,945 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

When you’re combining | and ==, there is a shortcut: %in%. It keeps rows where the variable equals one of the values on the right. Instead of the code above, you can write:

```

flights |>
  filter(month %in% c(1, 2))

```

```

# A tibble: 51,955 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     542           540           2     923           850
4  2013     1     1     544           545          -1    1004          1022
5  2013     1     1     554           600          -6     812           837
6  2013     1     1     554           558          -4     740           728
7  2013     1     1     555           600          -5     913           854
8  2013     1     1     557           600          -3     709           723
9  2013     1     1     557           600          -3     838           846
10 2013     1     1     558           600          -2     753           745
# i 51,945 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

```

flights |>
  filter(month %in% c(1, 2, 3, 4))

```

```
# A tibble: 109,119 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     542           540           2     923           850
4  2013     1     1     544           545          -1    1004          1022
5  2013     1     1     554           600          -6     812           837
6  2013     1     1     554           558          -4     740           728
7  2013     1     1     555           600          -5     913           854
8  2013     1     1     557           600          -3     709           723
9  2013     1     1     557           600          -3     838           846
10 2013     1     1     558           600          -2     753           745
# i 109,109 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Flights going to Houston OR Chicago:

```
flights |>
  filter(dest %in% c("IAH", "ORD"))
```

```
# A tibble: 24,481 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     554           558          -4     740           728
4  2013     1     1     558           600          -2     753           745
5  2013     1     1     608           600           8     807           735
6  2013     1     1     623           627          -4     933           932
7  2013     1     1     629           630          -1     824           810
8  2013     1     1     656           700          -4     854           850
9  2013     1     1     709           700           9     852           832
10 2013     1     1     715           713           2     911           850
# i 24,471 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Flights of United Airlines and American Airlines:

```
flights |>
  filter(carrier %in% c("UA", "AA"))
```

```
# A tibble: 91,394 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     542           540           2     923           850
4  2013     1     1     554           558          -4     740           728
5  2013     1     1     558           600          -2     753           745
6  2013     1     1     558           600          -2     924           917
7  2013     1     1     558           600          -2     923           937
8  2013     1     1     559           600          -1     941           910
9  2013     1     1     559           600          -1     854           902
10 2013     1     1     606           610          -4     858           910
# i 91,384 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
```

```
# tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
# hour <dbl>, minute <dbl>, time_hour <dtm>
```

It reads as “keep rows where month is IN this list of values”. Also note that numbers never get quotes, but text always does.

To save the new data frame that dplyr has created, you need to assign it with <-:

```
jan1 <- flights |>
  filter(month == 1, day == 1)
```

```
jan1
```

```
# A tibble: 842 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2     830             819
2  2013     1     1     533             529           4     850             830
3  2013     1     1     542             540           2     923             850
4  2013     1     1     544             545          -1    1004            1022
5  2013     1     1     554             600          -6     812             837
6  2013     1     1     554             558          -4     740             728
7  2013     1     1     555             600          -5     913             854
8  2013     1     1     557             600          -3     709             723
9  2013     1     1     557             600          -3     838             846
10 2013     1     1     558             600          -2     753             745
# i 832 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

It works on other data too:

```
penguins |>
  filter(species == "Adelie")
```

```
# A tibble: 152 × 8
  species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
  <fct>   <fct>         <dbl>         <dbl>         <int>         <int>
1 Adelie Torgersen     39.1           18.7           181           3750
2 Adelie Torgersen     39.5           17.4           186           3800
3 Adelie Torgersen     40.3           18           195           3250
4 Adelie Torgersen     NA             NA             NA             NA
5 Adelie Torgersen     36.7           19.3           193           3450
6 Adelie Torgersen     39.3           20.6           190           3650
7 Adelie Torgersen     38.9           17.8           181           3625
8 Adelie Torgersen     39.2           19.6           195           4675
9 Adelie Torgersen     34.1           18.1           193           3475
10 Adelie Torgersen     42            20.2           190           4250
# i 142 more rows
# i 2 more variables: sex <fct>, year <int>
```

Saving a data frame to a file:

```
write_csv(jan1, "jan2.csv")
```

How to commit to GitHub in the Terminal:

```
git add .
git commit -m "describe what you did"
git push
```

3.2.2 Common mistakes

(Notes to come.)

3.2.3 arrange()

`arrange()` sorts your data from lowest to highest (or highest to lowest) based on a column you choose.

If you provide more than one column name, each additional column is used to break ties in the values of the preceding columns. For instance:

```
flights |>
  arrange(year, month, day, dep_time)
```

```
# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     542           540           2     923           850
4  2013     1     1     544           545          -1    1004          1022
5  2013     1     1     554           600          -6     812           837
6  2013     1     1     554           558          -4     740           728
7  2013     1     1     555           600          -5     913           854
8  2013     1     1     557           600          -3     709           723
9  2013     1     1     557           600          -3     838           846
10 2013     1     1     558           600          -2     753           745
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Each column only kicks in when the previous one results in a tie.

```
dest_filter <- flights |>
  arrange(year, month, dest)
```

This sorts flights chronologically and then alphabetically by destination within each month. You'd see all January flights going to destinations starting with A, B, C and so on.

```
write.csv(dest_filter, "dest_filter.csv")
```

Which flights left earliest in the day?

```
flights |>
  arrange(dep_time)
```

```
# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1    13         1           2249           72     108           2357
2  2013     1    31         1           2100          181     124           2225
3  2013    11    13         1           2359           2     442           440
4  2013    12    16         1           2359           2     447           437
5  2013    12    20         1           2359           2     430           440
6  2013    12    26         1           2359           2     437           440
7  2013    12    30         1           2359           2     441           437
8  2013     2    11         1           2100          181     111           2225
9  2013     2    24         1           2245           76     121           2354
10 2013     3     8         1           2355           6     431           440
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
```

```
# tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
# hour <dbl>, minute <dbl>, time_hour <dtm>
```

To re-order by a column in descending (big-to-small) order, use `desc()`. This orders from most to least delayed:

```
flights |>
  arrange(desc(dep_delay))
```

```
# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     9     641           900         1301    1242          1530
2  2013     6    15    1432          1935         1137    1607          2120
3  2013     1    10    1121          1635         1126    1239          1810
4  2013     9    20    1139          1845         1014    1457          2210
5  2013     7    22     845          1600         1005    1044          1815
6  2013     4    10    1100          1900          960    1342          2211
7  2013     3    17    2321           810          911     135          1020
8  2013     6    27     959          1900          899    1236          2226
9  2013     7    22    2257           759          898     121          1026
10 2013    12     5     756          1700          896    1058          2020
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Sort flights chronologically by year, month and day, and within each day show the most delayed flights first:

```
flights |>
  arrange(year, month, day, desc(dep_delay))
```

```
# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     848          1835          853    1001          1950
2  2013     1     1    2343          1724          379     314          1938
3  2013     1     1    1815          1325          290    2120          1542
4  2013     1     1    2205          1720          285     46          2040
5  2013     1     1    1842          1422          260    1958          1535
6  2013     1     1    2115          1700          255    2330          1920
7  2013     1     1    2006          1630          216    2230          1848
8  2013     1     1    2312          2000          192     21          2110
9  2013     1     1    1942          1705          157    2124          1830
10 2013     1     1    1938          1703          155    2109          1823
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

3.2.4 `distinct()`

`distinct()` removes duplicate rows from the data, keeping only unique rows.

```
flights |>
  distinct()
```

```
# A tibble: 336,776 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     542           540           2     923           850
4  2013     1     1     544           545          -1    1004          1022
5  2013     1     1     554           600          -6     812           837
6  2013     1     1     554           558          -4     740           728
7  2013     1     1     555           600          -5     913           854
8  2013     1     1     557           600          -3     709           723
9  2013     1     1     557           600          -3     838           846
10 2013     1     1     558           600          -2     753           745
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

The difference from `filter()`:

```
# keep only Houston flights
flights |>
  filter(dest == "IAH")
```

```
# A tibble: 7,198 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     623           627          -4     933           932
4  2013     1     1     728           732          -4    1041          1038
5  2013     1     1     739           739           0    1104          1038
6  2013     1     1     908           908           0    1228          1219
7  2013     1     1    1028          1026           2    1350          1339
8  2013     1     1    1044          1045          -1    1352          1351
9  2013     1     1    1114           900        134    1447          1222
10 2013     1     1    1205          1200           5    1503          1505
# i 7,188 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
# keep only unique destinations
flights |>
  distinct(dest)
```

```
# A tibble: 105 × 1
  dest
  <chr>
1 IAH
2 MIA
3 BQN
4 ATL
5 ORD
6 FLL
7 IAD
8 MCO
9 PBI
10 TPA
# i 95 more rows
```

```
flights |>
  distinct(dest) |>
  nrow()
```

```
[1] 105
```

```
penguins |>
  distinct(bill_depth_mm) |>
  nrow()
```

```
[1] 81
```

Finding unique pairs of dest and origin:

```
flights |>
  distinct(dest, origin)
```

```
# A tibble: 224 × 2
  dest origin
<chr> <chr>
1 IAH EWR
2 IAH LGA
3 MIA JFK
4 BQN JFK
5 ATL LGA
6 ORD EWR
7 FLL EWR
8 IAD LGA
9 MCO JFK
10 ORD LGA
# i 214 more rows
```

```
flights |>
  distinct(dest, origin, carrier)
```

```
# A tibble: 439 × 3
  dest origin carrier
<chr> <chr> <chr>
1 IAH EWR UA
2 IAH LGA UA
3 MIA JFK AA
4 BQN JFK B6
5 ATL LGA DL
6 ORD EWR UA
7 FLL EWR B6
8 IAD LGA EV
9 MCO JFK B6
10 ORD LGA AA
# i 429 more rows
```

`distinct()` removes duplicate **combinations**, not necessarily duplicate values in each column individually. So `distinct(origin, dest)` doesn't give unique origins and unique destinations separately; it gives unique *pairs* of them.

To keep the other columns when filtering for unique rows, use the `.keep_all = TRUE` option:

```
flights |>
  distinct(dest, origin, carrier, .keep_all = TRUE)
```

```
# A tibble: 439 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2     830             819
2  2013     1     1     533             529           4     850             830
3  2013     1     1     542             540           2     923             850
4  2013     1     1     544             545          -1    1004            1022
5  2013     1     1     554             600          -6     812             837
6  2013     1     1     554             558          -4     740             728
7  2013     1     1     555             600          -5     913             854
8  2013     1     1     557             600          -3     709             723
9  2013     1     1     557             600          -3     838             846
10 2013     1     1     558             600          -2     753             745
# i 429 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
flights |>
  distinct(origin, dest, .keep_all = TRUE) |>
  nrow()
```

```
[1] 224
```

`.keep_all = TRUE` is just saying “yes, I want unique origin-dest combinations, but don’t throw away the rest of my data”.

`distinct()` only shows you the unique combinations. It doesn’t tell you how many times each one appears:

```
flights |>
  distinct(origin, dest)
```

```
# A tibble: 224 × 2
  origin dest
  <chr>   <chr>
1 EWR    IAH
2 LGA    IAH
3 JFK    MIA
4 JFK    BQN
5 LGA    ATL
6 EWR    ORD
7 EWR    FLL
8 LGA    IAD
9 JFK    MCO
10 LGA    ORD
# i 214 more rows
```

`count()` tells you how many times each combination appears:

```
flights |>
  count(origin, dest)
```

```
# A tibble: 224 × 3
  origin dest     n
  <chr>   <chr> <int>
1 EWR    ALB     439
2 EWR    ANC       8
3 EWR    ATL    5022
4 EWR    AUS     968
5 EWR    AVL     265
```

```
6 EWR    BDL    443
7 EWR    BNA    2336
8 EWR    BOS    5327
9 EWR    BQN    297
10 EWR   BTV    931
# i 214 more rows
```

Adding `sort = TRUE` arranges by most common first:

```
flights |>
  count(origin, dest, sort = TRUE)
```

```
# A tibble: 224 × 3
  origin dest      n
  <chr>  <chr> <int>
1 JFK    LAX    11262
2 LGA    ATL    10263
3 LGA    ORD     8857
4 JFK    SFO     8204
5 LGA    CLT     6168
6 EWR    ORD     6100
7 JFK    BOS     5898
8 LGA    MIA     5781
9 JFK    MCO     5464
10 EWR    BOS     5327
# i 214 more rows
```

```
flights |>
  count(dest, sort = TRUE)
```

```
# A tibble: 105 × 2
  dest      n
  <chr> <int>
1 ORD   17283
2 ATL   17215
3 LAX   16174
4 BOS   15508
5 MCO   14082
6 CLT   14064
7 SFO   13331
8 FLL   12055
9 MIA   11728
10 DCA    9705
# i 95 more rows
```

3.2.5 Exercises

1. In a single pipeline for each condition, find all flights that meet the condition.

Had an arrival delay of two or more hours:

```
flights |>
  filter(arr_delay >= 120) |>
  arrange(desc(arr_delay))
```

```
# A tibble: 10,200 × 9
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>    <int>         <int>
1  2013     1     9     641           900         1301    1242           1530
2  2013     6    15    1432          1935         1137    1607           2120
```

```

3 2013 1 10 1121 1635 1126 1239 1810
4 2013 9 20 1139 1845 1014 1457 2210
5 2013 7 22 845 1600 1005 1044 1815
6 2013 4 10 1100 1900 960 1342 2211
7 2013 3 17 2321 810 911 135 1020
8 2013 7 22 2257 759 898 121 1026
9 2013 12 5 756 1700 896 1058 2020
10 2013 5 3 1133 2055 878 1250 2215
# i 10,190 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

Flew to Houston (IAH or HOU):

```

flights |>
  filter(dest %in% c("IAH", "HOU"))

```

```

# A tibble: 9,313 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     623           627          -4     933           932
4  2013     1     1     728           732          -4    1041          1038
5  2013     1     1     739           739           0    1104          1038
6  2013     1     1     908           908           0    1228          1219
7  2013     1     1    1028          1026           2    1350          1339
8  2013     1     1    1044          1045          -1    1352          1351
9  2013     1     1    1114           900        134    1447          1222
10 2013     1     1    1205          1200           5    1503          1505
# i 9,303 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

or

```

flights |>
  filter(dest == "IAH" | dest == "HOU")

```

```

# A tibble: 9,313 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     623           627          -4     933           932
4  2013     1     1     728           732          -4    1041          1038
5  2013     1     1     739           739           0    1104          1038
6  2013     1     1     908           908           0    1228          1219
7  2013     1     1    1028          1026           2    1350          1339
8  2013     1     1    1044          1045          -1    1352          1351
9  2013     1     1    1114           900        134    1447          1222
10 2013     1     1    1205          1200           5    1503          1505
# i 9,303 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

Were operated by United, American, or Delta:

```
flights |>
  filter(carrier %in% c("UA", "AA", "DL"))
```

```
# A tibble: 139,504 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     542           540           2     923           850
4  2013     1     1     554           600          -6     812           837
5  2013     1     1     554           558          -4     740           728
6  2013     1     1     558           600          -2     753           745
7  2013     1     1     558           600          -2     924           917
8  2013     1     1     558           600          -2     923           937
9  2013     1     1     559           600          -1     941           910
10 2013     1     1     559           600          -1     854           902
# i 139,494 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Departed in summer (July, August, and September):

```
flights |>
  filter(month %in% c(7, 8, 9))
```

```
# A tibble: 86,326 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     7     1       1           2029         212     236           2359
2  2013     7     1       2           2359         3      344           344
3  2013     7     1      29           2245        104     151           1
4  2013     7     1     43           2130        193     322           14
5  2013     7     1     44           2150        174     300           100
6  2013     7     1     46           2051        235     304           2358
7  2013     7     1     48           2001        287     308           2305
8  2013     7     1     58           2155        183     335           43
9  2013     7     1    100           2146        194     327           30
10 2013     7     1    100           2245        135     337           135
# i 86,316 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Arrived more than two hours late but didn't leave late:

```
flights |>
  filter(arr_delay > 120, dep_delay == 0)
```

```
# A tibble: 3 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013    10     7    1350           1350           0     1736           1526
2  2013     5    23    1810           1810           0     2208           2000
3  2013     7     1     905           905           0     1443           1223
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Even better:

```
flights |>
  filter(arr_delay >= 120 & dep_delay <= 0)
```

```
# A tibble: 29 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1    27    1419           1420         -1     1754           1550
2  2013    10     7    1350           1350          0     1736           1526
3  2013    10     7    1357           1359         -2     1858           1654
4  2013    10    16     657             700         -3     1258           1056
5  2013    11     1     658             700         -2     1329           1015
6  2013     3    18    1844           1847         -3          39           2219
7  2013     4    17    1635           1640         -5     2049           1845
8  2013     4    18     558             600         -2     1149            850
9  2013     4    18     655             700         -5     1213            950
10 2013     5    22    1827           1830         -3     2217           2010
# i 19 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Using `dep_delay <= 0` is better because “didn’t leave late” also includes flights that left early (a negative delay).

Were delayed by at least an hour, but made up over 30 minutes in flight:

```
flights |>
  filter(dep_delay >= 60, dep_delay - arr_delay > 30)
```

```
# A tibble: 1,844 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1    2205           1720        285         46           2040
2  2013     1     1    2326           2130        116        131            18
3  2013     1     3    1503           1221        162       1803           1555
4  2013     1     3    1839           1700         99       2056           1950
5  2013     1     3    1850           1745         65       2148           2120
6  2013     1     3    1941           1759        102       2246           2139
7  2013     1     3    1950           1845         65       2228           2227
8  2013     1     3    2015           1915         60       2135           2111
9  2013     1     3    2257           2000        177         45           2224
10 2013     1     4    1917           1700        137       2135           1950
# i 1,834 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Making up time means the arrival delay was less than the departure delay by more than 30 minutes.

2. Sort `flights` to find the flights with the longest departure delays. Find the flights that left earliest in the morning.

```
flights |>
  arrange(desc(dep_delay))
```

```
# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     9     641             900       1301       1242           1530
2  2013     6    15    1432           1935       1137       1607           2120
3  2013     1    10    1121           1635       1126       1239           1810
4  2013     9    20    1139           1845       1014       1457           2210
```

```

5 2013 7 22 845 1600 1005 1044 1815
6 2013 4 10 1100 1900 960 1342 2211
7 2013 3 17 2321 810 911 135 1020
8 2013 6 27 959 1900 899 1236 2226
9 2013 7 22 2257 759 898 121 1026
10 2013 12 5 756 1700 896 1058 2020
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

```

flights |>
  arrange(dep_time)

```

```

# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1    13         1           2249          72     108           2357
2  2013     1    31         1           2100         181     124           2225
3  2013    11    13         1           2359          2     442           440
4  2013    12    16         1           2359          2     447           437
5  2013    12    20         1           2359          2     430           440
6  2013    12    26         1           2359          2     437           440
7  2013    12    30         1           2359          2     441           437
8  2013     2    11         1           2100         181     111           2225
9  2013     2    24         1           2245          76     121           2354
10 2013     3     8         1           2355          6     431           440
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

or

```

flights |>
  arrange(time_hour)

```

```

# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515          2     830           819
2  2013     1     1     533           529          4     850           830
3  2013     1     1     542           540          2     923           850
4  2013     1     1     544           545         -1    1004          1022
5  2013     1     1     554           558         -4     740           728
6  2013     1     1     559           559          0     702           706
7  2013     1     1     554           600         -6     812           837
8  2013     1     1     555           600         -5     913           854
9  2013     1     1     557           600         -3     709           723
10 2013     1     1     557           600         -3     838           846
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

`time_hour` might be a better column in some ways, because it stores the time properly as a real date-time object, without the gaps that clock-style times like `dep_time` have.

3. Sort `flights` to find the fastest flights. (Hint: try including a math calculation inside of your function.)

```
flights |>
  arrange(desc(distance / air_time))
```

```
# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     5    25    1709           1700          9    1923           1937
2  2013     7     2    1558           1513         45    1745           1719
3  2013     5    13    2040           2025        15    2225           2226
4  2013     3    23    1914           1910         4    2045           2043
5  2013     1    12    1559           1600        -1    1849           1917
6  2013    11    17     650            655        -5    1059           1150
7  2013     2    21    2355           2358        -3     412            438
8  2013    11    17     759            800        -1    1212           1255
9  2013    11    16    2003           1925        38      17             36
10 2013    11    16    2349           2359       -10     402            440
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Or, if “fastest” means shortest flight duration:

```
flights |>
  arrange(air_time)
```

```
# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1    16    1355           1315         40    1442           1411
2  2013     4    13     537            527         10     622            628
3  2013    12     6     922            851         31    1021           954
4  2013     2     3    2153           2129         24    2247           2224
5  2013     2     5    1303           1315        -12    1342           1411
6  2013     2    12    2123           2130         -7    2211           2225
7  2013     3     2    1450           1500        -10    1547           1608
8  2013     3     8    2026           1935         51    2131           2056
9  2013     3    18    1456           1329         87    1533           1426
10 2013     3    19    2226           2145         41    2305           2246
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

4. Was there a flight on every day of 2013?

```
flights |>
  distinct(year, month, day) |>
  count()
```

```
# A tibble: 1 × 1
   n
   <int>
1  365
```

or

```
flights |>
  distinct(year, month, day) |>
  nrow()
```

```
[1] 365
```

5. Which flights traveled the farthest distance? Which traveled the least distance?

```
flights |>
  arrange(desc(distance))
```

```
# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     857           900          -3    1516           1530
2  2013     1     2     909           900           9    1525           1530
3  2013     1     3     914           900          14    1504           1530
4  2013     1     4     900           900           0    1516           1530
5  2013     1     5     858           900          -2    1519           1530
6  2013     1     6    1019           900          79    1558           1530
7  2013     1     7    1042           900         102    1620           1530
8  2013     1     8     901           900           1    1504           1530
9  2013     1     9     641           900        1301    1242           1530
10 2013     1    10     859           900          -1    1449           1530
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

and

```
flights |>
  arrange(distance)
```

```
# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     7    27      NA           106          NA      NA           245
2  2013     1     3    2127          2129          -2    2222           2224
3  2013     1     4    1240          1200          40    1333           1306
4  2013     1     4    1829          1615         134    1937           1721
5  2013     1     4    2128          2129          -1    2218           2224
6  2013     1     5    1155          1200          -5    1241           1306
7  2013     1     6    2125          2129          -4    2224           2224
8  2013     1     7    2124          2129          -5    2212           2224
9  2013     1     8    2127          2130          -3    2304           2225
10 2013     1     9    2126          2129          -3    2217           2224
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

6. Does it matter what order you used `filter()` and `arrange()` if you're using both? Why/why not? Think about the results and how much work the functions would have to do.

The order doesn't matter for the final result, because `filter()` changes which rows are included, while `arrange()` only changes their order. However, the order can matter for efficiency. It is usually better to `filter()` first, because then `arrange()` has fewer rows to sort.

3.3 Columns

3.3.1 `mutate()`

`mutate()` lets you create new columns using math or logic on columns that already exist.

To compute the `gain` (how much time a delayed flight made up in the air) and the `speed` in miles per hour:

```
flights |>
  mutate(gain = dep_delay - arr_delay,
         speed = distance / air_time * 60)
```

```
# A tibble: 336,776 × 21
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2       830           819
2  2013     1     1     533             529           4       850           830
3  2013     1     1     542             540           2       923           850
4  2013     1     1     544             545          -1      1004          1022
5  2013     1     1     554             600          -6       812           837
6  2013     1     1     554             558          -4       740           728
7  2013     1     1     555             600          -5       913           854
8  2013     1     1     557             600          -3       709           723
9  2013     1     1     557             600          -3       838           846
10 2013     1     1     558             600          -2       753           745
# i 336,766 more rows
# i 13 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>, gain <dbl>, speed <dbl>
```

Create a TRUE/FALSE column: was the flight late?

```
flights |>
  mutate(is_late = dep_delay > 0)
```

```
# A tibble: 336,776 × 20
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2       830           819
2  2013     1     1     533             529           4       850           830
3  2013     1     1     542             540           2       923           850
4  2013     1     1     544             545          -1      1004          1022
5  2013     1     1     554             600          -6       812           837
6  2013     1     1     554             558          -4       740           728
7  2013     1     1     555             600          -5       913           854
8  2013     1     1     557             600          -3       709           723
9  2013     1     1     557             600          -3       838           846
10 2013     1     1     558             600          -2       753           745
# i 336,766 more rows
# i 12 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>, is_late <lgl>
```

Minutes to hours:

```
flights |>
  mutate(air_time_hours = air_time / 60)
```

```
# A tibble: 336,776 × 20
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2       830           819
2  2013     1     1     533             529           4       850           830
3  2013     1     1     542             540           2       923           850
4  2013     1     1     544             545          -1      1004          1022
5  2013     1     1     554             600          -6       812           837
6  2013     1     1     554             558          -4       740           728
7  2013     1     1     555             600          -5       913           854
```

```

8 2013 1 1 557 600 -3 709 723
9 2013 1 1 557 600 -3 838 846
10 2013 1 1 558 600 -2 753 745
# i 336,766 more rows
# i 12 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>, air_time_hours <dbl>

```

```

flights |>
  mutate(is_late = dep_delay > 0) |>
  filter(is_late)

```

```

# A tibble: 128,432 × 20
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2     830             819
2  2013     1     1     533             529           4     850             830
3  2013     1     1     542             540           2     923             850
4  2013     1     1     601             600           1     844             850
5  2013     1     1     608             600           8     807             735
6  2013     1     1     611             600          11     945             931
7  2013     1     1     613             610           3     925             921
8  2013     1     1     623             610          13     920             915
9  2013     1     1     632             608          24     740             728
10 2013     1     1     644             636           8     931             940
# i 128,422 more rows
# i 12 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>, is_late <lgl>

```

By default, `mutate()` adds new columns on the right-hand side of your dataset, which makes it difficult to see what's happening above. Use the `.before` argument to add the variables to the left-hand side instead:

```

flights |>
  mutate(
    gain = dep_delay - arr_delay,
    speed = distance / air_time * 60,
    .before = 1
  )

```

```

# A tibble: 336,776 × 21
  gain speed year month   day dep_time sched_dep_time dep_delay arr_time
  <dbl> <dbl> <int> <int> <int>   <int>         <int>         <dbl>   <int>
1   -9  370.  2013     1     1     517             515           2     830
2  -16  374.  2013     1     1     533             529           4     850
3  -31  408.  2013     1     1     542             540           2     923
4   17  517.  2013     1     1     544             545          -1    1004
5   19  394.  2013     1     1     554             600          -6     812
6  -16  288.  2013     1     1     554             558          -4     740
7  -24  404.  2013     1     1     555             600          -5     913
8   11  259.  2013     1     1     557             600          -3     709
9    5  405.  2013     1     1     557             600          -3     838
10 -10  319.  2013     1     1     558             600          -2     753
# i 336,766 more rows
# i 12 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier <chr>,
#   flight <int>, tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>,
#   distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dtm>

```

You can also use `.after` to add after a variable. In both `.before` and `.after` you can use the variable name instead of a position.

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    speed = distance / air_time * 60,
    .after = month
  )
```

```
# A tibble: 336,776 × 21
  year month  gain speed  day dep_time sched_dep_time dep_delay arr_time
  <int> <int> <dbl> <dbl> <int> <int>          <int>      <dbl>    <int>
1  2013     1    -9  370.     1     517            515         2      830
2  2013     1   -16  374.     1     533            529         4      850
3  2013     1   -31  408.     1     542            540         2      923
4  2013     1    17  517.     1     544            545        -1     1004
5  2013     1    19  394.     1     554            600        -6      812
6  2013     1   -16  288.     1     554            558        -4      740
7  2013     1   -24  404.     1     555            600        -5      913
8  2013     1    11  259.     1     557            600        -3      709
9  2013     1     5  405.     1     557            600        -3      838
10 2013     1   -10  319.     1     558            600        -2      753
# i 336,766 more rows
# i 12 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier <chr>,
#   flight <int>, tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>,
#   distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dtm>
```

You can control which variables are kept with the `.keep` argument. A useful value is "used", which keeps only the columns that were involved in or created by the `mutate()` step:

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    hours = air_time / 60,
    gain_per_hour = gain / hours,
    .keep = "used"
  )
```

```
# A tibble: 336,776 × 6
  dep_delay arr_delay air_time  gain hours gain_per_hour
  <dbl>      <dbl>    <dbl> <dbl> <dbl>      <dbl>
1         2        11     227    -9  3.78      -2.38
2         4        20     227   -16  3.78      -4.23
3         2        33     160   -31  2.67     -11.6
4        -1       -18     183    17  3.05       5.57
5        -6       -25     116    19  1.93       9.83
6        -4        12     150   -16  2.5       -6.4
7        -5        19     158   -24  2.63     -9.11
8        -3       -14      53    11  0.883     12.5
9        -3        -8     140     5  2.33       2.14
10       -2         8     138   -10  2.3      -4.35
# i 336,766 more rows
```

3.3.2 select()

`select()` lets you rapidly zoom in on a useful subset, using operations based on the names of the variables. Select columns by name:

```
flights |>
  select(year, day, month)
```

```
# A tibble: 336,776 × 3
  year   day month
  <int> <int> <int>
1  2013     1     1
2  2013     1     1
3  2013     1     1
4  2013     1     1
5  2013     1     1
6  2013     1     1
7  2013     1     1
8  2013     1     1
9  2013     1     1
10 2013     1     1
# i 336,766 more rows
```

Select all columns between year and carrier (inclusive):

```
flights |>
  select(year:carrier)
```

```
# A tibble: 336,776 × 10
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2       830           819
2  2013     1     1     533             529           4       850           830
3  2013     1     1     542             540           2       923           850
4  2013     1     1     544             545          -1      1004          1022
5  2013     1     1     554             600          -6       812           837
6  2013     1     1     554             558          -4       740           728
7  2013     1     1     555             600          -5       913           854
8  2013     1     1     557             600          -3       709           723
9  2013     1     1     557             600          -3       838           846
10 2013     1     1     558             600          -2       753           745
# i 336,766 more rows
# i 2 more variables: arr_delay <dbl>, carrier <chr>
```

Select all columns except those from year to dep_delay (inclusive):

```
flights |>
  select(!year:dep_delay)
```

```
# A tibble: 336,776 × 13
  arr_time sched_arr_time arr_delay carrier flight tailnum origin dest
  <int>         <int>         <dbl> <chr>   <int> <chr>   <chr> <chr>
1     830           819         11 UA      1545 N14228 EWR   IAH
2     850           830         20 UA      1714 N24211 LGA   IAH
3     923           850         33 AA      1141 N619AA JFK   MIA
4    1004          1022        -18 B6       725 N804JB JFK   BQN
5     812           837        -25 DL       461 N668DN LGA   ATL
6     740           728         12 UA      1696 N39463 EWR   ORD
7     913           854         19 B6       507 N516JB EWR   FLL
8     709           723        -14 EV      5708 N829AS LGA   IAD
9     838           846         -8 B6       79 N593JB JFK   MCO
10    753           745          8 AA       301 N3ALAA LGA   ORD
# i 336,766 more rows
# i 5 more variables: air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
#   time_hour <dtm>
```

Select all columns that are characters:

```
flights |>
  select(where(is.character))
```

```
# A tibble: 336,776 × 4
  carrier tailnum origin dest
<chr>    <chr>    <chr> <chr>
1 UA      N14228   EWR   IAH
2 UA      N24211   LGA   IAH
3 AA      N619AA    JFK   MIA
4 B6      N804JB     JFK   BQN
5 DL      N668DN     LGA   ATL
6 UA      N39463     EWR   ORD
7 B6      N516JB     EWR   FLL
8 EV      N829AS     LGA   IAD
9 B6      N593JB     JFK   MCO
10 AA     N3ALAA     LGA   ORD
# i 336,766 more rows
```

You can rename variables as you `select()` them by using `=`. The new name appears on the left-hand side of the `=`, and the old variable appears on the right-hand side:

```
flights |>
  select(tail_num = tailnum)
```

```
# A tibble: 336,776 × 1
  tail_num
<chr>
1 N14228
2 N24211
3 N619AA
4 N804JB
5 N668DN
6 N39463
7 N516JB
8 N829AS
9 N593JB
10 N3ALAA
# i 336,766 more rows
```

```
flights |>
  select(tailnum)
```

```
# A tibble: 336,776 × 1
  tailnum
<chr>
1 N14228
2 N24211
3 N619AA
4 N804JB
5 N668DN
6 N39463
7 N516JB
8 N829AS
9 N593JB
10 N3ALAA
# i 336,766 more rows
```

3.3.3 rename()

It always takes the form `rename(new_name = old_name)`.

If you want to keep all the existing variables and just rename a few, use `rename()` instead of `select()`:

```
flights |>
  rename(tail_number = tailnum,
         destination = dest)
```

```
# A tibble: 336,776 × 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515           2     830           819
2  2013     1     1     533           529           4     850           830
3  2013     1     1     542           540           2     923           850
4  2013     1     1     544           545          -1    1004          1022
5  2013     1     1     554           600          -6     812           837
6  2013     1     1     554           558          -4     740           728
7  2013     1     1     555           600          -5     913           854
8  2013     1     1     557           600          -3     709           723
9  2013     1     1     557           600          -3     838           846
10 2013     1     1     558           600          -2     753           745
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tail_number <chr>, origin <chr>, destination <chr>, air_time <dbl>,
#   distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dtm>
```

3.3.4 relocate()

`relocate()` moves variables around. You might want to collect related variables together or move important variables to the front. By default, it moves variables to the front:

```
flights |>
  relocate(time_hour, air_time)
```

```
# A tibble: 336,776 × 19
   time_hour          air_time year month   day dep_time sched_dep_time
   <dtm>          <dbl> <int> <int> <int>   <int>         <int>
1 2013-01-01 05:00:00      227  2013     1     1     517           515
2 2013-01-01 05:00:00      227  2013     1     1     533           529
3 2013-01-01 05:00:00      160  2013     1     1     542           540
4 2013-01-01 05:00:00      183  2013     1     1     544           545
5 2013-01-01 06:00:00      116  2013     1     1     554           600
6 2013-01-01 05:00:00      150  2013     1     1     554           558
7 2013-01-01 06:00:00      158  2013     1     1     555           600
8 2013-01-01 06:00:00       53  2013     1     1     557           600
9 2013-01-01 06:00:00      140  2013     1     1     557           600
10 2013-01-01 06:00:00      138  2013     1     1     558           600
# i 336,766 more rows
# i 12 more variables: dep_delay <dbl>, arr_time <int>, sched_arr_time <int>,
#   arr_delay <dbl>, carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
#   dest <chr>, distance <dbl>, hour <dbl>, minute <dbl>
```

You can also specify where to put them using the `.before` and `.after` arguments, just like in `mutate()`:

```
flights |>
  relocate(year:dep_time, .after = time_hour)
```

```
# A tibble: 336,776 × 19
   sched_dep_time dep_delay arr_time sched_arr_time arr_delay carrier flight
```

```

      <int>    <dbl>    <int>      <int>    <dbl> <chr>    <int>
1         515         2        830        819    11 UA      1545
2         529         4        850        830    20 UA      1714
3         540         2        923        850    33 AA      1141
4         545        -1       1004       1022   -18 B6      725
5         600        -6        812        837   -25 DL      461
6         558        -4        740        728    12 UA     1696
7         600        -5        913        854    19 B6      507
8         600        -3        709        723   -14 EV     5708
9         600        -3        838        846    -8 B6       79
10        600        -2        753        745     8 AA     301
# i 336,766 more rows
# i 12 more variables: tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>,
#   distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dtm>, year <int>,
#   month <int>, day <int>, dep_time <int>

```

```

flights |>
  relocate(starts_with("arr"), .before = year)

```

```

# A tibble: 336,776 × 19
  arr_time arr_delay year month   day dep_time sched_dep_time dep_delay
  <int>      <dbl> <int> <int> <int>   <int>      <int>      <dbl>
1      830         11  2013     1     1     517         515         2
2      850         20  2013     1     1     533         529         4
3      923         33  2013     1     1     542         540         2
4     1004        -18  2013     1     1     544         545        -1
5      812        -25  2013     1     1     554         600        -6
6      740         12  2013     1     1     554         558        -4
7      913         19  2013     1     1     555         600        -5
8      709        -14  2013     1     1     557         600        -3
9      838         -8  2013     1     1     557         600        -3
10     753          8  2013     1     1     558         600        -2
# i 336,766 more rows
# i 11 more variables: sched_arr_time <int>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

```

flights |>
  relocate(starts_with("dep"), .after = year)

```

```

# A tibble: 336,776 × 19
  year dep_time dep_delay month   day sched_dep_time arr_time sched_arr_time
  <int>   <int>      <dbl> <int> <int>      <int>      <int>      <int>
1  2013     517         2     1     1         515      830         819
2  2013     533         4     1     1         529      850         830
3  2013     542         2     1     1         540      923         850
4  2013     544        -1     1     1         545     1004        1022
5  2013     554        -6     1     1         600      812         837
6  2013     554        -4     1     1         558      740         728
7  2013     555        -5     1     1         600      913         854
8  2013     557        -3     1     1         600      709         723
9  2013     557        -3     1     1         600      838         846
10 2013     558        -2     1     1         600      753         745
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

```
flights |>
  relocate(ends_with("time"), .before = year)
```

```
# A tibble: 336,776 × 19
  dep_time sched_dep_time arr_time sched_arr_time air_time year month   day
  <int>      <int>      <int>      <int>      <dbl> <int> <int> <int>
1     517         515         830         819      227  2013     1     1
2     533         529         850         830      227  2013     1     1
3     542         540         923         850      160  2013     1     1
4     544         545        1004        1022      183  2013     1     1
5     554         600         812         837      116  2013     1     1
6     554         558         740         728      150  2013     1     1
7     555         600         913         854      158  2013     1     1
8     557         600         709         723       53  2013     1     1
9     557         600         838         846      140  2013     1     1
10    558         600         753         745      138  2013     1     1
# i 336,766 more rows
# i 11 more variables: dep_delay <dbl>, arr_delay <dbl>, carrier <chr>,
#   flight <int>, tailnum <chr>, origin <chr>, dest <chr>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

3.3.5 Exercises

1. Compare `dep_time`, `sched_dep_time`, and `dep_delay`. How would you expect those three numbers to be related?

```
flights |>
  select(dep_time, sched_dep_time, dep_delay)
```

```
# A tibble: 336,776 × 3
  dep_time sched_dep_time dep_delay
  <int>      <int>      <dbl>
1     517         515          2
2     533         529          4
3     542         540          2
4     544         545         -1
5     554         600         -6
6     554         558         -4
7     555         600         -5
8     557         600         -3
9     557         600         -3
10    558         600         -2
# i 336,766 more rows
```

`dep_delay` is the difference between the actual departure time and the scheduled departure time. Knowing any two of the three variables should conceptually let you work out the third, although `dep_time` and `sched_dep_time` are stored as clock times (like 517 for 5:17) rather than as simple numbers of minutes.

2. Brainstorm as many ways as possible to select `dep_time`, `dep_delay`, `arr_time`, and `arr_delay` from `flights`.

```
flights |>
  select(dep_time, dep_delay, arr_time, arr_delay)
```

```
# A tibble: 336,776 × 4
  dep_time dep_delay arr_time arr_delay
  <int>      <dbl>      <int>      <dbl>
1     517          2         830         11
2     533          4         850         20
3     542          2         923         33
```

```

4      544      -1    1004     -18
5      554      -6     812     -25
6      554      -4     740      12
7      555      -5     913      19
8      557      -3     709     -14
9      557      -3     838      -8
10     558      -2     753       8
# i 336,766 more rows

```

```

flights |>
  select(starts_with("dep"), starts_with("arr"))

```

```

# A tibble: 336,776 × 4
  dep_time dep_delay arr_time arr_delay
  <int>     <dbl>   <int>     <dbl>
1     517         2     830         11
2     533         4     850         20
3     542         2     923         33
4     544        -1    1004        -18
5     554        -6     812        -25
6     554        -4     740         12
7     555        -5     913         19
8     557        -3     709        -14
9     557        -3     838         -8
10    558        -2     753          8
# i 336,766 more rows

```

```

flights |>
  select(contains("dep_"), contains("arr_"))

```

```

# A tibble: 336,776 × 6
  dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
  <int>         <int>     <dbl>   <int>         <int>         <dbl>
1     517         515         2     830           819          11
2     533         529         4     850           830          20
3     542         540         2     923           850          33
4     544         545        -1    1004          1022         -18
5     554         600        -6     812           837         -25
6     554         558        -4     740           728          12
7     555         600        -5     913           854          19
8     557         600        -3     709           723         -14
9     557         600        -3     838           846          -8
10    558         600        -2     753           745           8
# i 336,766 more rows

```

3. What happens if you specify the name of the same variable multiple times in a `select()` call?

```

flights |>
  select(year, day, arr_time, year)

```

```

# A tibble: 336,776 × 3
  year   day arr_time
  <int> <int>   <int>
1  2013     1     830
2  2013     1     850
3  2013     1     923
4  2013     1    1004
5  2013     1     812
6  2013     1     740

```

```

7 2013      1      913
8 2013      1      709
9 2013      1      838
10 2013     1      753
# i 336,766 more rows

```

`select()` doesn't duplicate the variable, and it respects the order in which you first select the columns.

4. What does the `any_of()` function do? Why might it be helpful in conjunction with this vector?

```
variables <- c("year", "month", "day", "dep_delay", "arr_delay")
```

`any_of()` lets you use a character vector of column names inside `select()` to select those columns. It's useful when the names are stored in a vector rather than typed directly into `select()`, and it can safely ignore names that don't exist.

For example, with a vector that includes a column that doesn't exist:

```

variables <- c(
  "year",
  "month",
  "day",
  "dep_delay",
  "arr_delay",
  "banana"
)

flights |>
  select(any_of(variables))

```

```

# A tibble: 336,776 × 5
   year month   day dep_delay arr_delay
  <int> <int> <int>    <dbl>    <dbl>
1  2013     1     1         2         11
2  2013     1     1         4         20
3  2013     1     1         2         33
4  2013     1     1        -1        -18
5  2013     1     1        -6        -25
6  2013     1     1        -4         12
7  2013     1     1        -5         19
8  2013     1     1        -3        -14
9  2013     1     1        -3         -8
10 2013     1     1        -2          8
# i 336,766 more rows

```

Without `any_of()`, the missing column causes an error:

```

flights |>
  select(variables)

```

```

Error in `select()`:
! Can't select columns that don't exist.
* Column `banana` doesn't exist.

```

5. Does the result of running the following code surprise you? How do the select helpers deal with upper and lower case by default? How can you change that default?

```

flights |>
  select(contains("TIME"))

```

```

# A tibble: 336,776 × 6
  dep_time sched_dep_time arr_time sched_arr_time air_time time_hour

```

```

      <int>      <int>      <int>      <int>      <dbl> <dtm>
1      517      515      830      819      227 2013-01-01 05:00:00
2      533      529      850      830      227 2013-01-01 05:00:00
3      542      540      923      850      160 2013-01-01 05:00:00
4      544      545     1004     1022     183 2013-01-01 05:00:00
5      554      600      812      837     116 2013-01-01 06:00:00
6      554      558      740      728     150 2013-01-01 05:00:00
7      555      600      913      854     158 2013-01-01 06:00:00
8      557      600      709      723      53 2013-01-01 06:00:00
9      557      600      838      846     140 2013-01-01 06:00:00
10     558      600      753      745     138 2013-01-01 06:00:00
# i 336,766 more rows

```

```

flights |>
  select(starts_with("DEP"))

```

```

# A tibble: 336,776 × 2
  dep_time dep_delay
  <int>      <dbl>
1      517          2
2      533          4
3      542          2
4      544         -1
5      554         -6
6      554         -4
7      555         -5
8      557         -3
9      557         -3
10     558         -2
# i 336,766 more rows

```

The `select()` helper functions such as `contains()` and `starts_with()` are **case-insensitive** by default. To change that default, use `ignore.case = FALSE`:

```

flights |>
  select(contains("TIME", ignore.case = FALSE))

```

```

# A tibble: 336,776 × 0

```

Now R looks for “TIME” exactly as written. But the actual `flights` columns contain “time” in lowercase, so nothing matches:

```

flights |>
  select(contains("time", ignore.case = FALSE))

```

```

# A tibble: 336,776 × 6
  dep_time sched_dep_time arr_time sched_arr_time air_time time_hour
  <int>      <int>      <int>      <int>      <dbl> <dtm>
1      517      515      830      819      227 2013-01-01 05:00:00
2      533      529      850      830      227 2013-01-01 05:00:00
3      542      540      923      850      160 2013-01-01 05:00:00
4      544      545     1004     1022     183 2013-01-01 05:00:00
5      554      600      812      837     116 2013-01-01 06:00:00
6      554      558      740      728     150 2013-01-01 05:00:00
7      555      600      913      854     158 2013-01-01 06:00:00
8      557      600      709      723      53 2013-01-01 06:00:00
9      557      600      838      846     140 2013-01-01 06:00:00
10     558      600      753      745     138 2013-01-01 06:00:00
# i 336,766 more rows

```

`ignore.case = FALSE` tells R to be case sensitive.

6. Rename `air_time` to `air_time_min` to indicate units of measurement and move it to the beginning of the data frame.

```
flights |>
  rename(air_time_min = air_time) |>
  relocate(air_time_min)
```

```
# A tibble: 336,776 × 19
  air_time_min year month   day dep_time sched_dep_time dep_delay arr_time
    <dbl> <int> <int> <int>   <int>         <int>         <dbl>   <int>
1      227  2013     1     1     517           515         2     830
2      227  2013     1     1     533           529         4     850
3      160  2013     1     1     542           540         2     923
4      183  2013     1     1     544           545        -1    1004
5      116  2013     1     1     554           600        -6     812
6      150  2013     1     1     554           558        -4     740
7      158  2013     1     1     555           600        -5     913
8       53  2013     1     1     557           600        -3     709
9      140  2013     1     1     557           600        -3     838
10     138  2013     1     1     558           600        -2     753
# i 336,766 more rows
# i 11 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier <chr>,
#   flight <int>, tailnum <chr>, origin <chr>, dest <chr>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

7. Why doesn't the following work, and what does the error mean?

```
flights |>
  select(tailnum) |>
  arrange(arr_delay)
```

```
Error in `arrange()` :
! In argument: `..1 = arr_delay`.
Caused by error:
! object 'arr_delay' not found
```

`select()` creates a new data frame that only has `tailnum`, so `arr_delay` no longer exists and there is nothing to arrange by.

3.4 The pipe

(Notes to come.)

3.5 Groups

Grouping divides a dataset into groups for meaningful analysis.

3.5.1 `group_by()`

```
flights |>
  group_by(month)
```

```
# A tibble: 336,776 × 19
# Groups:   month [12]
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515         2     830         819
2  2013     1     1     533           529         4     850         830
3  2013     1     1     542           540         2     923         850
```

```

4 2013 1 1 544 545 -1 1004 1022
5 2013 1 1 554 600 -6 812 837
6 2013 1 1 554 558 -4 740 728
7 2013 1 1 555 600 -5 913 854
8 2013 1 1 557 600 -3 709 723
9 2013 1 1 557 600 -3 838 846
10 2013 1 1 558 600 -2 753 745
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

`group_by()` doesn't change the data, but if you look closely at the output, it says the data is "grouped by" month (Groups: month [12]). This means subsequent operations will now work "by month".

3.5.2 summarize()

If it is used to calculate a single summary statistic, `summarize()` reduces the data frame to a single row for each group.

```

flights |>
  group_by(month) |>
  summarise(avg_delay = mean(dep_delay))

```

```

# A tibble: 12 × 2
  month avg_delay
  <int>   <dbl>
1     1      NA
2     2      NA
3     3      NA
4     4      NA
5     5      NA
6     6      NA
7     7      NA
8     8      NA
9     9      NA
10    10      NA
11    11      NA
12    12      NA

```

This throws up NAs because there are some missing values in the dataset. Use `na.rm = TRUE` to remove them:

```

flights |>
  group_by(month) |>
  summarise(avg_delay = mean(dep_delay, na.rm = TRUE))

```

```

# A tibble: 12 × 2
  month avg_delay
  <int>   <dbl>
1     1    10.0
2     2    10.8
3     3    13.2
4     4    13.9
5     5    13.0
6     6    20.8
7     7    21.7
8     8    12.6
9     9     6.72
10    10     6.24

```

```
11 11 5.44
12 12 16.6
```

```
flights |>
  group_by(origin) |>
  summarise(avg_delay = mean(dep_delay, na.rm = TRUE))
```

```
# A tibble: 3 × 2
  origin avg_delay
<chr>    <dbl>
1 EWR      15.1
2 JFK      12.1
3 LGA      10.3
```

There can be multiple summaries in a single `summarise()` call, for instance `n()`, which returns the number of rows in each group:

```
flights |>
  group_by(month) |>
  summarise(
    avg_arr_delay = mean(arr_delay, na.rm = TRUE),
    n = n()
  )
```

```
# A tibble: 12 × 3
  month avg_arr_delay    n
<int>    <dbl> <int>
1     1      6.13 27004
2     2      5.61 24951
3     3      5.81 28834
4     4     11.2 28330
5     5      3.52 28796
6     6     16.5 28243
7     7     16.7 29425
8     8      6.04 29327
9     9     -4.02 27574
10    10     -0.167 28889
11    11      0.461 27268
12    12     14.9 28135
```

```
flights |>
  group_by(month) |>
  summarise(
    avg_distance = mean(distance, na.rm = TRUE),
    n = n()
  )
```

```
# A tibble: 12 × 3
  month avg_distance    n
<int>    <dbl> <int>
1     1    1007. 27004
2     2    1001. 24951
3     3    1012. 28834
4     4    1039. 28330
5     5    1041. 28796
6     6    1057. 28243
7     7    1059. 29425
8     8    1062. 29327
9     9    1041. 27574
```

```
10 10 1039. 28889
11 11 1050. 27268
12 12 1065. 28135
```

3.5.3 The slice_ functions

These mainly work with rows based on position. For example, the first five rows of flights:

```
flights |>
  slice(1:5)
```

```
# A tibble: 5 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517             515           2     830           819
2  2013     1     1     533             529           4     850           830
3  2013     1     1     542             540           2     923           850
4  2013     1     1     544             545          -1    1004          1022
5  2013     1     1     554             600          -6     812           837
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

or row 32:

```
flights |>
  slice(32)
```

```
# A tibble: 1 × 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     623             610          13     920           915
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

They also let you extract specific rows within each group:

- `df |> slice_head(n = 1)` takes the first row from each group.
- `df |> slice_tail(n = 1)` takes the last row in each group.
- `df |> slice_min(x, n = 1)` takes the row with the smallest value of column `x`.
- `df |> slice_max(x, n = 1)` takes the row with the largest value of column `x`.
- `df |> slice_sample(n = 1)` takes one random row.

To find the flights that are most delayed upon arrival at each destination:

```
flights |>
  group_by(dest) |>
  slice_max(arr_delay, n = 1) |>
  relocate(dest)
```

```
# A tibble: 108 × 19
# Groups:   dest [105]
  dest year month   day dep_time sched_dep_time dep_delay arr_time
  <chr> <int> <int> <int>   <int>         <int>         <dbl>   <int>
1 ABQ  2013     7    22    2145             2007          98     132
2 ACK  2013     7    23    1139             800         219    1250
3 ALB  2013     1    25     123             2000         323     229
4 ANC  2013     8    17    1740             1625          75    2042
5 ATL  2013     7    22    2257             759         898     121
```

```

6 AUS      2013      7    10    2056          1505      351    2347
7 AVL      2013      8    13    1156          832      204    1417
8 BDL      2013      2    21    1728          1316      252    1839
9 BGR      2013     12      1    1504          1056      248    1628
10 BHM     2013      4    10      25          1900      325     136
# i 98 more rows
# i 11 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier <chr>,
#   flight <int>, tailnum <chr>, origin <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

Note that there are 105 destinations but we get 108 rows here. What's up? `slice_min()` and `slice_max()` keep tied values, so `n = 1` means "give us all rows with the highest value". If you want exactly one row per group, set `with_ties = FALSE`:

```

flights |>
  group_by(dest) |>
  slice_max(arr_delay, n = 1, with_ties = FALSE)

```

```

# A tibble: 105 × 19
# Groups:   dest [105]
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     7    22    2145           2007           98     132           2259
2  2013     7    23    1139           800          219     1250           909
3  2013     1    25     123           2000          323     229           2101
4  2013     8    17    1740           1625           75     2042           2003
5  2013     7    22    2257           759          898     121           1026
6  2013     7    10    2056          1505          351     2347           1758
7  2013     8    13    1156           832          204     1417           1029
8  2013     2    21    1728          1316          252     1839           1413
9  2013    12     1    1504          1056          248     1628           1230
10 2013     4    10      25          1900          325     136           2045
# i 95 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

To find the flights most delayed at departure at each origin:

```

flights |>
  group_by(origin) |>
  slice_max(dep_delay, n = 1) |>
  relocate(dep_delay)

```

```

# A tibble: 3 × 19
# Groups:   origin [3]
  dep_delay year month   day dep_time sched_dep_time arr_time sched_arr_time
   <dbl> <int> <int> <int>   <int>         <int>         <int>         <int>
1    1126  2013     1    10    1121           1635     1239           1810
2    1301  2013     1     9     641           900     1242           1530
3     911  2013     3    17    2321           810     135           1020
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>

```

Which tailnum had the most flights?

```

flights |>
  count(tailnum, sort = TRUE) |>
  slice(1:5)

```

```
# A tibble: 5 × 2
  tailnum      n
  <chr>    <int>
1 <NA>    2512
2 N725MQ    575
3 N722MQ    513
4 N723MQ    507
5 N711MQ    486
```

3.5.4 Grouping by multiple variables

You can create groups using more than one variable. For example, we could make a group for each date:

```
daily <- flights |>
  group_by(year, month, day)
```

```
daily |>
  summarise(number_of_rows = n())
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by year, month, and day.
i Output is grouped by year and month.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(year, month, day))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 365 × 4
# Groups:   year, month [12]
  year month   day number_of_rows
  <int> <int> <int>         <int>
1  2013     1     1             842
2  2013     1     2             943
3  2013     1     3             914
4  2013     1     4             915
5  2013     1     5             720
6  2013     1     6             832
7  2013     1     7             933
8  2013     1     8             899
9  2013     1     9             902
10 2013     1    10             932
# i 355 more rows
```

When you summarize a tibble grouped by more than one variable, each summary peels off the last group, in this case day. It also shows a message. To suppress it, use `.groups = "drop_last"`:

```
daily_flights <- daily |>
  summarise(
    n = n(),
    .groups = "drop_last"
  )
```

Alternatively, change the default behaviour by setting a different value, e.g. `"drop"` to drop all grouping or `"keep"` to preserve the same groups:

```
daily |>
  summarise(n = n(), .groups = "keep") |>
  arrange(desc(n))
```

```
# A tibble: 365 × 4
# Groups:   year, month, day [365]
   year month  day    n
  <int> <int> <int> <int>
1  2013    11   27  1014
2  2013     7   11  1006
3  2013     7    8  1004
4  2013     7   10  1004
5  2013    12    2  1004
6  2013     7   18  1003
7  2013     7   25  1003
8  2013     7   12  1002
9  2013     7    9  1001
10 2013     7   17  1001
# i 355 more rows
```

3.5.5 Ungrouping

You can also remove grouping from a data frame without using `summarize()`:

```
daily |>
  ungroup()
```

```
# A tibble: 336,776 × 19
   year month  day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>      <dbl>      <int>         <int>
1  2013     1    1     517             515         2         830           819
2  2013     1    1     533             529         4         850           830
3  2013     1    1     542             540         2         923           850
4  2013     1    1     544             545        -1        1004          1022
5  2013     1    1     554             600        -6         812           837
6  2013     1    1     554             558        -4         740           728
7  2013     1    1     555             600        -5         913           854
8  2013     1    1     557             600        -3         709           723
9  2013     1    1     557             600        -3         838           846
10 2013     1    1     558             600        -2         753           745
# i 336,766 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

What happens when you summarize an ungrouped data frame?

```
daily |>
  ungroup() |>
  summarise(
    avg_delay = mean(dep_delay, na.rm = TRUE),
    flights = n()
  )
```

```
# A tibble: 1 × 2
  avg_delay flights
  <dbl>     <int>
1    12.6   336776
```

You get a single row back, because `dplyr` treats all the rows in an ungrouped data frame as belonging to one group. It is basically the `flights` data as one group:

```
flights |>
  summarise(
```

```

    av_delay = mean(dep_delay, na.rm = TRUE),
    flights = n()
  )

```

```

# A tibble: 1 × 2
  av_delay flights
  <dbl>   <int>
1    12.6  336776

```

3.5.6 .by

For instance:

```

flights |>
  summarise(
    avg_arr_delay = mean(arr_delay, na.rm = TRUE),
    total_flights = n(),
    .by = month
  )

```

```

# A tibble: 12 × 3
  month avg_arr_delay total_flights
  <int>         <dbl>         <int>
1     1          6.13          27004
2    10         -0.167          28889
3    11          0.461          27268
4    12         14.9           28135
5     2          5.61          24951
6     3          5.81          28834
7     4         11.2          28330
8     5          3.52          28796
9     6         16.5          28243
10    7         16.7          29425
11    8          6.04          29327
12    9         -4.02          27574

```

or

```

flights |>
  summarise(
    n = n(),
    .by = month
  )

```

```

# A tibble: 12 × 2
  month     n
  <int> <int>
1     1 27004
2    10 28889
3    11 27268
4    12 28135
5     2 24951
6     3 28834
7     4 28330
8     5 28796
9     6 28243
10    7 29425
11    8 29327
12    9 27574

```

This means: group the data by month, then summarise each group. With `.by`, you do both inside the one operation. `.by` is **per-operation grouping**.

You can also use `.by` with multiple variables:

```
flights |>
  summarise(
    avg_arr_delay = mean(arr_delay, na.rm = TRUE),
    total_flights = n(),
    .by = c(origin, dest)
  ) |>
  arrange(desc(avg_arr_delay))
```

```
# A tibble: 224 × 4
  origin dest avg_arr_delay total_flights
  <chr>  <chr>      <dbl>      <int>
1 EWR    CAE        44.6        104
2 EWR    TYS        41.2        323
3 EWR    TUL        33.7        315
4 EWR    OKC        30.6        346
5 EWR    JAC        29.9         23
6 EWR    RIC        25.8       1667
7 EWR    MKE        24.0       1094
8 EWR    DSM        23.5        411
9 EWR    MSN        23.2        354
10 EWR    PWM        23.0        770
# i 214 more rows
```

For this summary, we group the flights by the combination of `origin` AND `dest`. Each unique origin + dest combination is a group.

Keep in mind:

- `group_by()` = group first, then do things.
- `.by` = group while doing this particular thing.

3.5.7 Exercises

1. Which carrier has the worst average delays? Challenge: can you disentangle the effects of bad airports vs. bad carriers? Why/why not? (Hint: think about `flights |> group_by(carrier, dest) |> summarize(n())`.)

```
flights |>
  summarise(
    avg_dep_delay = mean(dep_delay, na.rm = TRUE),
    avg_arr_delay = mean(arr_delay, na.rm = TRUE),
    .by = carrier,
    total_flight = n()
  ) |>
  arrange(desc(avg_dep_delay))
```

```
# A tibble: 16 × 4
  carrier avg_dep_delay avg_arr_delay total_flight
  <chr>      <dbl>      <dbl>      <int>
1 F9         20.2        21.9        685
2 EV         20.0        15.8       54173
3 YV         19.0        15.6        601
4 FL         18.7        20.1       3260
5 WN         17.7         9.65      12275
6 9E         16.7         7.38      18460
7 B6         13.0         9.46      54635
8 VX         12.9         1.76       5162
9 00         12.6        11.9         32
```

10	UA	12.1	3.56	58665
11	MQ	10.6	10.8	26397
12	DL	9.26	1.64	48110
13	AA	8.59	0.364	32729
14	AS	5.80	-9.93	714
15	HA	4.90	-6.92	342
16	US	3.78	2.13	20536

You don't have to compute multiple means.

```
flights |>
  group_by(carrier, dest) |>
  summarize(n())
```

```
# A tibble: 314 × 3
# Groups:   carrier [16]
  carrier dest `n()`
  <chr>   <chr> <int>
1 9E     ATL     59
2 9E     AUS      2
3 9E     AVL     10
4 9E     BGR      1
5 9E     BNA    474
6 9E     BOS    914
7 9E     BTV      2
8 9E     BUF    833
9 9E     BWI    856
10 9E     CAE      3
# i 304 more rows
```

Part 2: the key is to look at carrier + destination together.

```
flights |>
  summarise(
    avg_delay = mean(dep_delay, na.rm = TRUE),
    n = n(),
    .by = c(carrier, dest)
  ) |>
  arrange(dest)
```

```
# A tibble: 314 × 4
  carrier dest avg_delay    n
  <chr>   <chr>   <dbl> <int>
1 B6     ABQ    13.7   254
2 B6     ACK     6.46  265
3 EV     ALB    23.6   439
4 UA     ANC    12.9     8
5 DL     ATL    10.4 10571
6 MQ     ATL     9.35  2322
7 FL     ATL    18.4  2337
8 EV     ATL    22.4  1764
9 9E     ATL     0.965   59
10 WN    ATL     2.34   59
# i 304 more rows
```

2. Find the flights that are most delayed upon departure from each destination.

```
flights |>
  group_by(dest) |>
  slice_max(dep_delay) |>
```

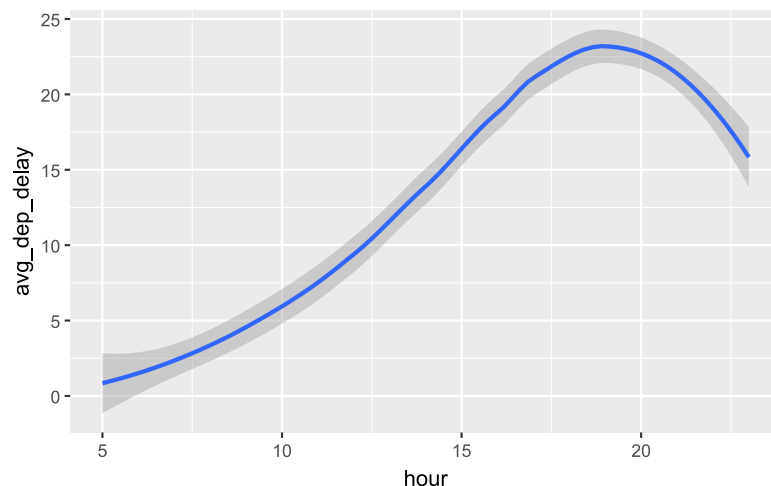
```
arrange(dest, dep_delay) |>
relocate(dep_delay, dest)
```

```
# A tibble: 105 × 19
# Groups:   dest [105]
  dep_delay dest    year month   day dep_time sched_dep_time arr_time
    <dbl> <chr>    <int> <int> <int> <int>         <int>    <int>
1      142 ABQ     2013    12    14    2223             2001     133
2      219 ACK     2013     7    23    1139             800     1250
3      323 ALB     2013     1    25     123             2000     229
4       75 ANC     2013     8    17    1740             1625    2042
5      898 ATL     2013     7    22    2257             759     121
6      351 AUS     2013     7    10    2056             1505    2347
7      222 AVL     2013     6    14    1158             816    1335
8      252 BDL     2013     2    21    1728             1316    1839
9      248 BGR     2013    12     1    1504             1056    1628
10     325 BHM     2013     4    10      25             1900     136
# i 95 more rows
# i 11 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier <chr>,
#   flight <int>, tailnum <chr>, origin <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

3. How do delays vary over the course of the day? Illustrate your answer with a plot.

Logic: group the flights by the hour they depart, calculate the average departure delay for each hour, and plot the averages to see how delays change throughout the day.

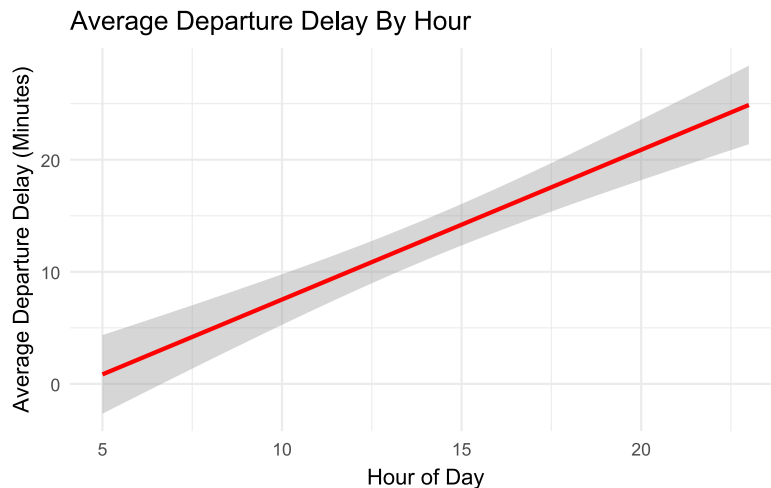
```
flights |>
  summarise(
    avg_dep_delay = mean(dep_delay, na.rm = TRUE),
    .by = hour
  ) |>
  ggplot(aes(hour, avg_dep_delay)) +
  geom_smooth()
```



A “go a little crazy” version:

```
flights |>
  summarise(
    avg_dep_delay = mean(dep_delay, na.rm = TRUE),
    .by = hour
  ) |>
  ggplot(aes(hour, avg_dep_delay)) +
  geom_smooth(method = "lm", colour = "red") +
```

```
labs(
  title = "Average Departure Delay By Hour",
  x = "Hour of Day",
  y = "Average Departure Delay (Minutes)"
) +
theme_minimal()
```



```
ggsave(filename = "avg_delay_plot.png")
```

4. What happens if you supply a negative `n` to `slice_min()` and friends?

```
flights |>
  group_by(origin) |>
  slice_min(arr_delay, n = -1, with_ties = FALSE) |>
  count()
```

```
# A tibble: 3 × 2
# Groups:   origin [3]
  origin      n
  <chr>   <int>
1 EWR    120834
2 JFK    111278
3 LGA    104661
```

For each origin, it finds the flight with the minimum `arr_delay`, removes it, and keeps everything else. That is what a negative `n` does: it means “remove that many of the smallest/largest rows”. `slice_head()` and `slice_tail()` also accept a negative `n`, but there the meaning is based on row position rather than on minimum/maximum values.

5. Explain what `count()` does in terms of the dplyr verbs you just learned. What does the `sort` argument to `count()` do?

`count()` is a shortcut for `group_by() + summarise(n = n())`. `sort = TRUE` sorts the resulting counts from largest to smallest.

```
flights |>
  count(origin)
```

```
# A tibble: 3 × 2
  origin      n
  <chr>   <int>
1 EWR    120835
2 JFK    111279
3 LGA    104662
```

This is basically:

```
flights |>
  group_by(origin) |>
  summarise(n = n())
```

```
# A tibble: 3 × 2
  origin      n
  <chr>   <int>
1 EWR    120835
2 JFK    111279
3 LGA    104662
```

With `sort = TRUE`:

```
flights |>
  count(origin, sort = TRUE)
```

```
# A tibble: 3 × 2
  origin      n
  <chr>   <int>
1 EWR    120835
2 JFK    111279
3 LGA    104662
```

6. Suppose we have the following tiny data frame:

```
df <- tibble(
  x = 1:5,
  y = c("a", "b", "a", "a", "b"),
  z = c("K", "K", "L", "L", "K")
)
```

a. Write down what you think the output will look like, then check if you were correct, and describe what `group_by()` does.

```
df |>
  group_by(y)
```

```
# A tibble: 5 × 3
# Groups:   y [2]
   x y     z
  <int> <chr> <chr>
1     1 a     K
2     2 b     K
3     3 a     L
4     4 a     L
5     5 b     K
```

`group_by()` does NOT collapse or rearrange the data. R now remembers “this data is grouped according to `y`”. So there are two groups: `y = "a"` (rows 1, 3, 4) and `y = "b"` (rows 2, 5). It says: “From now on, when I use a verb that works with groups, treat the data separately according to `y`.”

```
df |>
  group_by(y) |>
  summarise(n = n())
```

```
# A tibble: 2 × 2
  y      n
  <chr> <int>
1 a      3
2 b      2
```

`group_by(y)` doesn't change the rows or columns. It adds grouping information so that subsequent `dplyr` verbs can operate separately for each value of `y`.

b. Write down what you think the output will look like, then check if you were correct, and describe what `arrange()` does. Also comment on how it's different from the `group_by()` in part (a).

```
df |>
  arrange(y)
```

```
# A tibble: 5 × 3
      x y     z
  <int> <chr> <chr>
1     1 a     K
2     3 a     L
3     4 a     L
4     2 b     K
5     5 b     K
```

`arrange(y)` sorts the rows of the tibble according to the values of `y`, putting them in alphabetical order. The other variables move with their corresponding rows, so the values in each row stay together.

Unlike `group_by()`, `arrange()` actually changes the order of the rows. `group_by()` does not rearrange the data; it only adds grouping information for subsequent operations.

c. Write down what you think the output will look like, then check if you were correct, and describe what the pipeline does.

```
df |>
  group_by(y) |>
  summarise(mean_x = mean(x))
```

```
# A tibble: 2 × 2
  y     mean_x
  <chr>   <dbl>
1 a       2.67
2 b       3.5
```

The pipeline first groups the data by `y`, creating groups for `a` and `b`. `summarise()` then calculates the mean of `x` separately for each group, producing one row for each value of `y`. The pipe `|>` passes the result of each step to the next function.

d. Write down what you think the output will look like, then check if you were correct, and describe what the pipeline does. Then comment on what the message says.

```
df |>
  group_by(y, z) |>
  summarise(mean_x = mean(x))
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by y and z.
i Output is grouped by y.
i Use `summarise(groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(y, z))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 3 × 3
# Groups:   y [2]
  y     z     mean_x
  <chr> <chr>   <dbl>
1 a     K       1
2 a     L     3.5
3 b     K     3.5
```

`group_by(y, z)` tells `summarise()` to calculate the summaries for each y-z combination. After summarising, `summarise()` drops the last grouping level by default, leaving the result grouped by `y`. That is what the message says.

e. Write down what you think the output will look like, then check if you were correct, and describe what the pipeline does. How is the output different from the one in part (d)?

```
df |>
  group_by(y, z) |>
  summarize(mean_x = mean(x), .groups = "drop")
```

```
# A tibble: 3 × 3
  y     z   mean_x
  <chr> <chr> <dbl>
1 a     K     1
2 a     L    3.5
3 b     K    3.5
```

`.groups = "drop"` removes all grouping after `summarise()`. The actual summary results are the same as in part (d), but the resulting tibble is completely ungrouped. `.groups` doesn't change how the mean is calculated here; it controls what happens to the grouping after the calculation.

f. Write down what you think the outputs will look like, then check if you were correct, and describe what each pipeline does. How are the outputs of the two pipelines different?

```
df |>
  group_by(y, z) |>
  summarize(mean_x = mean(x))
```

```
# A tibble: 3 × 3
# Groups:   y [2]
  y     z   mean_x
  <chr> <chr> <dbl>
1 a     K     1
2 a     L    3.5
3 b     K    3.5
```

```
df |>
  group_by(y, z) |>
  mutate(mean_x = mean(x))
```

```
# A tibble: 5 × 4
# Groups:   y, z [3]
  x y     z   mean_x
  <int> <chr> <chr> <dbl>
1 1 a     K     1
2 2 b     K    3.5
3 3 a     L    3.5
4 4 a     L    3.5
5 5 b     K    3.5
```

`summarise()` reduces each group to one row, while `mutate()` keeps every original row and adds the group-level calculation to each row.

3.6 Case study: aggregates and sample size

If you haven't installed the Lahman package, run this once in the Console:

```
install.packages("Lahman")
```

```
library(Lahman)
```

```
View(LahmanData)
View(Teams)
View(Batting)
```

When you summarise data, always include `n()` so you know how many observations your conclusion is based on. A conclusion based on very little data is unreliable and potentially misleading. The baseball example shows this: a player's hit rate means very little if they only had a few attempts.

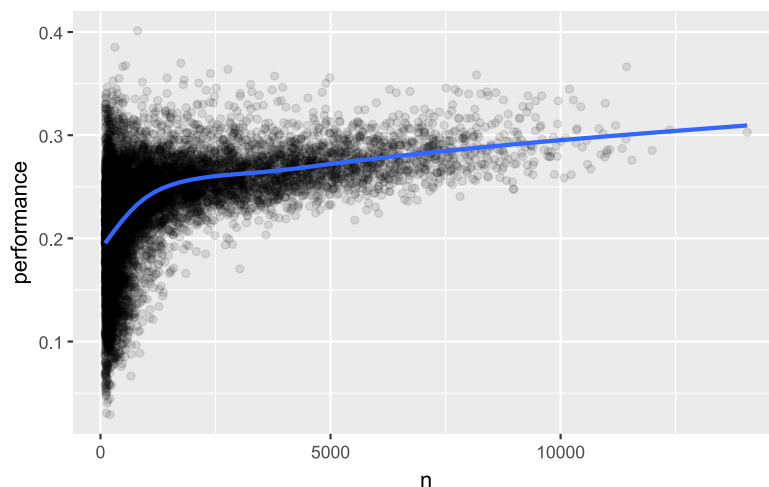
Here we compare the proportion of times a player gets a hit (H) with the number of times they try to put the ball in play (AB):

```
batters <- Lahman::Batting |>
  group_by(playerID) |>
  summarize(
    performance = sum(H, na.rm = TRUE) / sum(AB, na.rm = TRUE),
    n = sum(AB, na.rm = TRUE)
  )
```

The `::` basically means “take `Batting` from the `Lahman` package”.

When plotting performance against sample size, players with fewer observations show greater variation, while performance becomes more stable as the number of observations increases. There is also a positive correlation between performance and opportunities, because teams tend to give better-performing players more opportunities.

```
batters |>
  filter(n > 100) |>
  ggplot(aes(x = n, y = performance)) +
  geom_point(alpha = 1 / 10) +
  geom_smooth(se = FALSE)
```



Ranking players only by batting average can be misleading, because very high averages may be based on very few at-bats. Sample size should therefore be considered when interpreting or ranking performance.

```
batters |>
  arrange(desc(performance))
```

```
# A tibble: 24,011 × 3
  playerID performance    n
  <chr>         <dbl> <int>
1 abramge01         1     1
2 alberan01         1     1
3 averysk01         1     1
4 banisje01         1     1
5 barbesa01         1     2
6 bartocl01         1     1
```

```
7 bassdo01      1      1
8 bennedo01     1      1
9 birasst01     1      2
10 brews01      1      1
# i 24,001 more rows
```

4. Workflow: code style

If you haven't installed the `styler` package, run this once in the Console:

```
install.packages("styler")
```

```
library(tidyverse)
library(nycflights13)
```

The code in the “Strive for” and “Avoid” examples below is only shown, not run.

4.1 Names

Variable names (those created by `<-` and those created by `mutate()`) should use only lowercase letters, numbers, and `_`. Use `_` to separate words within a name.

```
# Strive for:
short_flights <- flights |>
  filter(air_time < 60)

# Avoid:
SHORTFLIGHTS <- flights |>
  filter(air_time < 60)
```

As a general rule of thumb, it's better to prefer long, descriptive names that are easy to understand rather than concise names that are fast to type.

4.2 Spaces

Put spaces on either side of mathematical operators apart from `^` (i.e. `+`, `-`, `==`, `<`, ...), and around the assignment operator (`<-`).

```
# Strive for
z <- (a + b)^2 / d

# Avoid
z<-( a + b ) ^ 2/d
```

Don't put spaces inside or outside parentheses for regular function calls. Always put a space after a comma, just like in standard English.

```
# Strive for
mean(x, na.rm = TRUE)

# Avoid
mean (x ,na.rm=TRUE)
```

You can add extra spaces if it improves alignment. For example, if you're creating multiple variables in `mutate()`, you might add spaces so that all the `=` line up. This makes the code easier to skim.

```
flights |>
  mutate(
    speed      = distance / air_time,
    dep_hour   = dep_time %/% 100,
    dep_minute = dep_time %/% 100
  )
```

4.3 Pipes

`|>` should always have a space before it and should typically be the last thing on a line.

```
# Strive for
flights |>
  filter(!is.na(arr_delay), !is.na(tailnum)) |>
  count(dest)

# Avoid
flights|>filter(!is.na(arr_delay), !is.na(tailnum))|>count(dest)
```

If the function you're piping into has named arguments (like `mutate()` or `summarize()`), put each argument on a new line. If the function doesn't have named arguments (like `select()` or `filter()`), keep everything on one line unless it doesn't fit, in which case put each argument on its own line.

```
# Strive for
flights |>
  group_by(tailnum) |>
  summarize(
    delay = mean(arr_delay, na.rm = TRUE),
    n = n()
  )

# Avoid
flights |>
  group_by(
    tailnum
  ) |>
  summarize(delay = mean(arr_delay, na.rm = TRUE), n = n())
```

After the first step of the pipeline, indent each line by two spaces. RStudio will automatically put the spaces in for you after a line break following a `|>`. If you're putting each argument on its own line, indent by an extra two spaces. Make sure `)` is on its own line, and un-indented to match the horizontal position of the function name.

```
# Strive for
flights |>
  group_by(tailnum) |>
  summarize(
    delay = mean(arr_delay, na.rm = TRUE),
    n = n()
  )

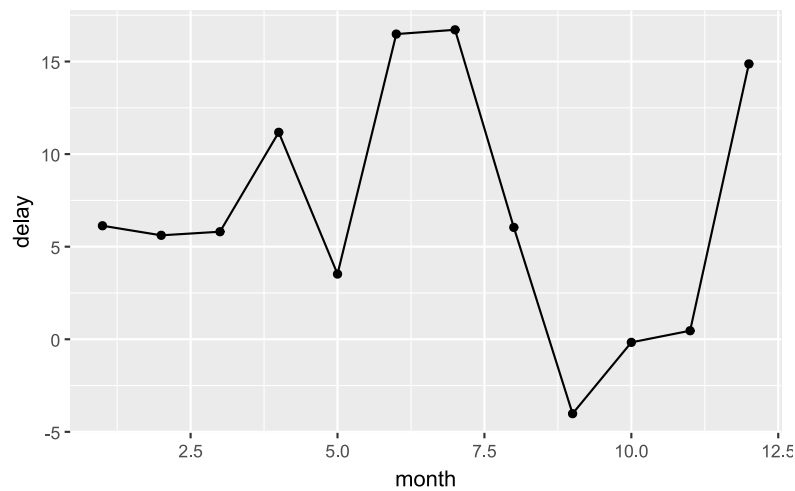
# Avoid: arguments and closing parenthesis indented too far
flights |>
  group_by(tailnum) |>
  summarize(
    delay = mean(arr_delay, na.rm = TRUE),
    n = n()
  )
```

4.4 ggplot2

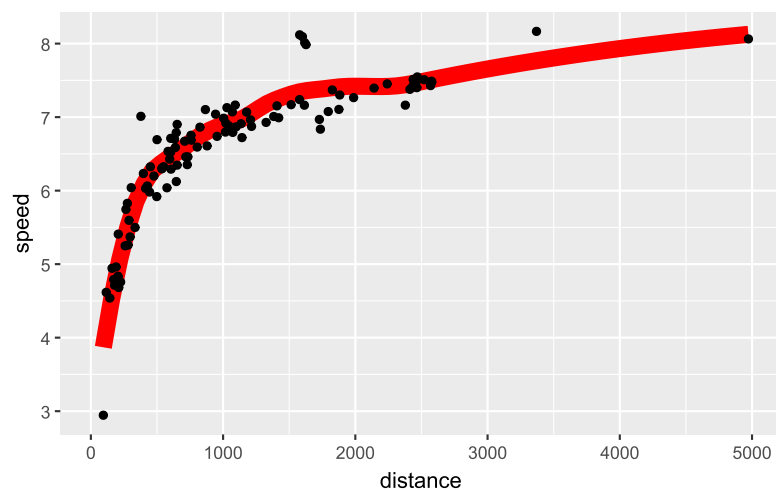
The same basic rules that apply to the pipe also apply to `ggplot2`; just treat `+` the same way as `|>`.

```
flights |>
  group_by(month) |>
  summarize(delay = mean(arr_delay, na.rm = TRUE)) |>
  ggplot(aes(x = month, y = delay)) +
```

```
geom_point() +
geom_line()
```



```
flights |>
  group_by(dest) |>
  summarize(
    distance = mean(distance),
    speed = mean(distance / air_time, na.rm = TRUE)
  ) |>
  ggplot(aes(x = distance, y = speed)) +
  geom_smooth(
    method = "loess",
    span = 0.5,
    se = FALSE,
    color = "red",
    linewidth = 4
  ) +
  geom_point()
```



4.5 Sectioning comments

(Notes to come.)

4.6 Exercises

1. Restyle the following pipelines following the guidelines above.

The messy versions:

```
flights|>filter(dest=="IAH")|>group_by(year,month,day)|>summarize(n=n(),
                                                                delay=mean(arr_delay,na.rm=TRUE))|
>filter(n>10)

flights|>filter(carrier=="UA",dest%in%c("IAH","HOU"),sched_dep_time>
0900,sched_arr_time<2000)|>group_by(flight)|>summarize(delay=mean(
arr_delay,na.rm=TRUE),cancelled=sum(is.na(arr_delay)),n=n())|
>filter(n>10)
```

The clean versions:

```
flights |>
  filter(dest == "IAH") |>
  group_by(year, month, day) |>
  summarise(n = n(), delay = mean(arr_delay, na.rm = TRUE)) |>
  filter(n > 10)
```

```
# A tibble: 365 × 5
# Groups:   year, month [12]
   year month   day     n delay
  <int> <int> <int> <int> <dbl>
1  2013     1     1     20  17.8
2  2013     1     2     20    7
3  2013     1     3     19  18.3
4  2013     1     4     20  -3.2
5  2013     1     5     13  20.2
6  2013     1     6     18   9.28
7  2013     1     7     19  -7.74
8  2013     1     8     19   7.79
9  2013     1     9     19  18.1
10 2013     1    10     19   6.68
# i 355 more rows
```

```
flights |>
  filter(
    carrier == "UA",
    dest %in% c("IAH", "HOU"),
    sched_dep_time > 0900,
    sched_arr_time < 2000
  ) |>
  group_by(flight) |>
  summarise(
    delay = mean(arr_delay, na.rm = TRUE),
    cancelled = sum(is.na(arr_delay)),
    n = n()
  ) |>
  filter(n > 10)
```

```
# A tibble: 74 × 4
   flight delay cancelled     n
  <int> <dbl>     <int> <int>
1    53  12.5         2    18
2   112  14.1         0    14
3   205  -1.71        0    14
4   235  -5.36        0    14
```

```
5    255 -9.47      0    15
6    268 38.6       1    15
7    292  6.57      0    21
8    318 10.7       1    20
9    337 20.1       2    21
10   370 17.5       0    11
# i 64 more rows
```


5. Data tidying

5.1 Introduction

5.1.1 Prerequisites

```
library(tidyverse)
```

5.2 Tidy data

You can represent the same underlying data in multiple ways. Tidy data is easier to read and work with.

There are three interrelated rules that make a dataset tidy:

1. Each variable is a column; each column is a variable.
2. Each observation is a row; each row is an observation.
3. Each value is a cell; each cell is a single value.

The tidyverse packages are designed to work with tidy data. Example:

```
table1
```

```
# A tibble: 6 × 4
  country    year cases population
  <chr>    <dbl> <dbl>      <dbl>
1 Afghanistan 1999    745  19987071
2 Afghanistan 2000   2666 20595360
3 Brazil      1999  37737 172006362
4 Brazil      2000  80488 174504898
5 China       1999 212258 1272915272
6 China       2000 213766 1280428583
```

Compute the rate per 10,000:

```
table1 |>
  mutate(rate = cases / population * 10000)
```

```
# A tibble: 6 × 5
  country    year cases population rate
  <chr>    <dbl> <dbl>      <dbl> <dbl>
1 Afghanistan 1999    745  19987071 0.373
2 Afghanistan 2000   2666 20595360 1.29
3 Brazil      1999  37737 172006362 2.19
4 Brazil      2000  80488 174504898 4.61
5 China       1999 212258 1272915272 1.67
6 China       2000 213766 1280428583 1.67
```

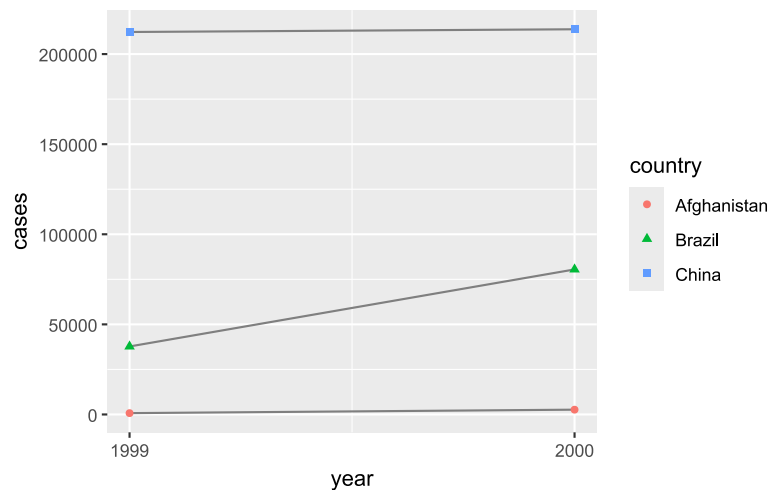
Total cases per year:

```
table1 |>
  summarise(
    total_cases = sum(cases),
    .by = year
  )
```

```
# A tibble: 2 × 2
  year total_cases
  <dbl>     <dbl>
1  1999     250740
2  2000     296920
```

Visualize changes over time:

```
ggplot(table1, aes(x = year, y = cases)) +
  geom_line(aes(group = country), color = "grey50") +
  geom_point(aes(color = country, shape = country)) +
  scale_x_continuous(breaks = c(1999, 2000)) # x-axis breaks at 1999 and 2000
```



5.2.1 Exercises

1. For each of the sample tables, describe what each observation and each column represents.

table1

```
# A tibble: 6 × 4
  country year cases population
  <chr>   <dbl> <dbl>     <dbl>
1 Afghanistan 1999    745  19987071
2 Afghanistan 2000   2666  20595360
3 Brazil      1999  37737  172006362
4 Brazil      2000  80488  174504898
5 China       1999 212258 1272915272
6 China       2000 213766 1280428583
```

table2

```
# A tibble: 12 × 4
  country year type count
  <chr>   <dbl> <chr>     <dbl>
1 Afghanistan 1999 cases      745
2 Afghanistan 1999 population 19987071
3 Afghanistan 2000 cases      2666
4 Afghanistan 2000 population 20595360
5 Brazil      1999 cases     37737
6 Brazil      1999 population 172006362
7 Brazil      2000 cases      80488
8 Brazil      2000 population 174504898
```

```

9 China      1999 cases      212258
10 China     1999 population 1272915272
11 China     2000 cases      213766
12 China     2000 population 1280428583

```

```
table3
```

```

# A tibble: 6 × 3
  country      year rate
  <chr>      <dbl> <chr>
1 Afghanistan 1999 745/19987071
2 Afghanistan 2000 2666/20595360
3 Brazil      1999 37737/172006362
4 Brazil      2000 80488/174504898
5 China       1999 212258/1272915272
6 China       2000 213766/1280428583

```

- **table1**: each row represents a country in a particular year. Cases and population are stored in separate columns.
- **table2**: each row represents a country, year, and measurement type. Cases and population are stored in one `count` column, which makes this a longer format.
- **table3**: each row represents a country in a particular year. Cases and population are combined into a character column called `rate`, which makes numerical calculations more difficult.

2. Sketch out the process you'd use to calculate the rate for **table2 and **table3**. You will need to perform four operations:**

- Extract the number of TB cases per country per year.
- Extract the matching population per country per year.
- Divide cases by population, and multiply by 10000.
- Store back in the appropriate place.

From ChatGPT:

For **table2**, I would group the data by country and year, extract the count for cases and the count for population, divide cases by population, multiply by 10,000, and store the result in a new `rate` column. For **table3**, I would first separate the `rate` column into cases and population, convert both values into numbers, calculate cases divided by population times 10,000, and replace the original text with the numerical rate.

The code for **table2**:

```

table2 |>
  pivot_wider(
    names_from = type,
    values_from = count
  ) |>
  mutate(
    rate = cases / population * 10000
  )

```

```

# A tibble: 6 × 5
  country      year cases population rate
  <chr>      <dbl> <dbl>      <dbl> <dbl>
1 Afghanistan 1999     745  19987071 0.373
2 Afghanistan 2000    2666  20595360 1.29
3 Brazil      1999   37737  172006362 2.19
4 Brazil      2000   80488  174504898 4.61
5 China       1999  212258  1272915272 1.67
6 China       2000  213766  1280428583 1.67

```

5.3 Lengthening data

Pivoting transforms data into a tidy form: variables in columns and observations in rows.

The tidyr functions for this are:

1. `pivot_longer()`
2. `pivot_wider()`

5.3.1 Data in column names

billboard

```
# A tibble: 317 × 79
  artist      track date.entered  wk1  wk2  wk3  wk4  wk5  wk6  wk7  wk8
  <chr>      <chr> <date>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 2 Pac      Baby... 2000-02-26    87    82    72    77    87    94    99    NA
2 2Ge+her    The ... 2000-09-02    91    87    92    NA    NA    NA    NA    NA
3 3 Doors D... Kryp... 2000-04-08    81    70    68    67    66    57    54    53
4 3 Doors D... Loser 2000-10-21    76    76    72    69    67    65    55    59
5 504 Boyz    Wobb... 2000-04-15    57    34    25    17    17    31    36    49
6 98^0       Give... 2000-08-19    51    39    34    26    26    19     2     2
7 A*Teens    Danc... 2000-07-08    97    97    96    95   100    NA    NA    NA
8 Aaliyah    I Do... 2000-01-29    84    62    51    41    38    35    35    38
9 Aaliyah    Try ... 2000-03-18    59    53    38    28    21    18    16    14
10 Adams, Yo... Open... 2000-08-26    76    76    74    69    68    67    61    58
# i 307 more rows
# i 68 more variables: wk9 <dbl>, wk10 <dbl>, wk11 <dbl>, wk12 <dbl>,
# wk13 <dbl>, wk14 <dbl>, wk15 <dbl>, wk16 <dbl>, wk17 <dbl>, wk18 <dbl>,
# wk19 <dbl>, wk20 <dbl>, wk21 <dbl>, wk22 <dbl>, wk23 <dbl>, wk24 <dbl>,
# wk25 <dbl>, wk26 <dbl>, wk27 <dbl>, wk28 <dbl>, wk29 <dbl>, wk30 <dbl>,
# wk31 <dbl>, wk32 <dbl>, wk33 <dbl>, wk34 <dbl>, wk35 <dbl>, wk36 <dbl>,
# wk37 <dbl>, wk38 <dbl>, wk39 <dbl>, wk40 <dbl>, wk41 <dbl>, wk42 <dbl>, ...
```

In the Billboard data, each row initially represents one song. The weekly rankings are spread across the columns `wk1` to `wk76`. The column names contain the week variable, while the cell values contain the rank variable. `pivot_longer()` converts these weekly columns into rows, so that each row represents one song's rank in one week.

```
billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    values_to = "position"
  )
```

```
# A tibble: 24,092 × 5
  artist track      date.entered week position
  <chr>   <chr>      <date>      <chr>    <dbl>
1 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk1      87
2 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk2      82
3 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk3      72
4 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk4      77
5 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk5      87
6 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk6      94
7 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk7      99
8 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk8      NA
9 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk9      NA
10 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk10     NA
# i 24,082 more rows
```

Or a different way of selecting the columns:

```
billboard |>
  pivot_longer(
    cols = !c(artist, track, date.entered),
    names_to = "week",
    values_to = "rank"
  )
```

```
# A tibble: 24,092 × 5
  artist track          date.entered week  rank
  <chr>  <chr>          <date>    <chr> <dbl>
1 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk1     87
2 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk2     82
3 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk3     72
4 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk4     77
5 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk5     87
6 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk6     94
7 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk7     99
8 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk8     NA
9 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk9     NA
10 2 Pac  Baby Don't Cry (Keep... 2000-02-26 wk10    NA
# i 24,082 more rows
```

`pivot_longer()` converts the columns beginning with `wk` into rows. The old column names are stored in a new `week` column, while the ranking values are stored in a new `rank` column.

- `cols` identifies the columns to pivot.
- `names_to` gives a name to the new column containing the old column names.
- `values_to` gives a name to the new column containing the old cell values.

Structural missing values occur because of how the data is organized, rather than because an expected value is unknown. In the Billboard data, `NA` means a song was not in the Top 100 during that week. `values_drop_na = TRUE` removes these unnecessary `NA` rows when using `pivot_longer()`.

```
billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    values_to = "position",
    names_to = "week",
    values_drop_na = TRUE
  )
```

```
# A tibble: 5,307 × 5
  artist track          date.entered week  position
  <chr>  <chr>          <date>    <chr>    <dbl>
1 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk1      87
2 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk2      82
3 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk3      72
4 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk4      77
5 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk5      87
6 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk6      94
7 2 Pac   Baby Don't Cry (Keep... 2000-02-26 wk7      99
8 2Ge+her The Hardest Part Of ... 2000-09-02 wk1      91
9 2Ge+her The Hardest Part Of ... 2000-09-02 wk2      87
10 2Ge+her The Hardest Part Of ... 2000-09-02 wk3      92
# i 5,297 more rows
```

To make computation easier, we can convert the values of `week` from character strings to numbers, using `mutate()` and `readr::parse_number()`:

```
billboard_longer <- billboard |>
  pivot_longer(
```

```
cols = starts_with("wk"),
names_to = "week",
values_to = "position",
values_drop_na = TRUE
) |>
mutate(week = parse_number(week))
```

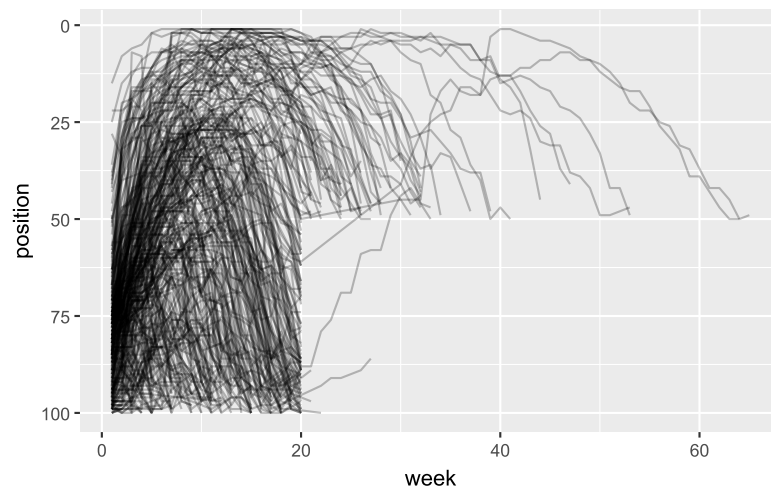
billboard_longer

```
# A tibble: 5,307 × 5
  artist track date.entered week position
  <chr> <chr> <date> <dbl> <dbl>
1 2 Pac Baby Don't Cry (Keep... 2000-02-26 1 87
2 2 Pac Baby Don't Cry (Keep... 2000-02-26 2 82
3 2 Pac Baby Don't Cry (Keep... 2000-02-26 3 72
4 2 Pac Baby Don't Cry (Keep... 2000-02-26 4 77
5 2 Pac Baby Don't Cry (Keep... 2000-02-26 5 87
6 2 Pac Baby Don't Cry (Keep... 2000-02-26 6 94
7 2 Pac Baby Don't Cry (Keep... 2000-02-26 7 99
8 2Ge+her The Hardest Part Of ... 2000-09-02 1 91
9 2Ge+her The Hardest Part Of ... 2000-09-02 2 87
10 2Ge+her The Hardest Part Of ... 2000-09-02 3 92
# i 5,297 more rows
```

`pivot_longer()` initially creates `week` as character values such as “wk1” and “wk2”. `mutate()` with `parse_number()` extracts the numbers and converts `week` to a numeric variable, making future calculations easier.

We can now visualize how song ranks vary over time:

```
billboard_longer |>
  ggplot(aes(week, position, group = track)) +
  geom_line(alpha = 0.25) +
  scale_y_reverse()
```



5.3.2 How does pivoting work?

Suppose we have 3 patients, each with 2 blood pressure readings:

```
df <- tribble(
  ~id, ~bp1, ~bp2,
  "A", 100, 120,
  "B", 140, 115,
```

```
"C", 120, 125
)
```

We want our new dataset to have three variables: `id` (which already exists), `measurement` (the column names), and `value` (the cell values). To achieve this, we pivot `df` longer:

```
df |>
  pivot_longer(
    cols = bp1:bp2,
    names_to = "measurement",
    values_to = "value"
  )
```

```
# A tibble: 6 × 3
  id      measurement value
<chr> <chr>      <dbl>
1 A      bp1         100
2 A      bp2         120
3 B      bp1         140
4 B      bp2         115
5 C      bp1         120
6 C      bp2         125
```


6. Summary

In summary, this book has no content whatsoever.

1 + 1

[1] 2

References

Bibliography